



Faculty of Software Engineering
Field: Information Technology

Illia Novikov 91651

**Overview and Simulation of a Smart Powertrain Control System for
Optimizing Energy Efficiency in Hybrid Electric Vehicles**

Engineering project work

Supervisor:
mgr inż. Przemysław Żarnecki

Wrocław 2025

Table of Contents

Abstract	4
List of Abbreviations	5
1. Introduction	6
1.1. Introduction of the problem	6
1.2. Objective	7
1.3. Focus on HEV	8
1.4. HEVs Architecture	9
1.5. Role of EMS	11
1.6. Effect of EMS	12
2. Aim of Project	13
2.1. Insights and Prospects	13
2.2. Issues and Solutions	14
3. Literature Overview	15
3.1. HEV Powertrain Architecture	15
3.1.1. ICE	15
3.1.2. Electric Machine (Motor/Generator)	16
3.1.3. Battery Energy Storage System (BESS)	17
3.1.4. Vehicle Longitudinal Dynamics	17
3.1.5. Controller (EMS)	18
3.2. HEV Control Strategies Overview	19
3.2.1. Rule-based	19
3.2.2. Optimization-based	21
3.2.3. Dynamic Rule-based	22
3.3. Driving Cycles and Data Used	23
3.4. Modelling Approaches	23
4. Application of Project	25
4.1. Planned Work	25
4.2. Outcomes	26
4.3. Practical Applications	27
4.3.1. Models Parameters	27
4.3.2. Vehicle Model	29
4.3.3. Motor Model	37

4.3.4.	Engine Model	42
4.3.5.	Battery Model.....	46
4.3.6.	Models Test	53
4.3.7.	Controller	55
4.4.	Simulation	58
4.4.1.	Synthetic Cycle	61
4.4.2.	WLTP Cycle (Real Life-like Cycle).....	65
4.5.	Comparison of Simulation Results	68
5.	Conclusion	70
	Bibliography	71

Abstract

Hybrid electric vehicles are becoming increasingly important in modern transportation, with their overall performance largely dependent on effective powertrain energy management. The energy management system (EMS) decides how power is split between the engine and electric motor while keeping the battery within a healthy state of charge. To achieve high efficiency under real driving conditions, the EMS must quickly respond to changing driver demands and make smart decisions that reduce both fuel consumption and electrical losses. This project overviews different types of EMS and implements a rule-based EMS within a detailed vehicle simulation to evaluate hybrid performance over different drive cycles, including synthetic and WLTP-like patterns. The simulation uses realistic models for the engine, motor, battery, and vehicle dynamics to assess fuel use, electrical energy consumption, and overall efficiency. Results show that the EMS effectively uses electric propulsion during low-load and idle phases, minimizing fuel consumption without sacrificing drivability. The simulated fuel consumption aligns closely with manufacturer WLTP data, confirming the model's accuracy. Additionally, related metrics provide deeper insight into hybrid operation. Although the simulation focuses on baseline controller settings, it highlights the potential benefits of more advanced predictive control methods for further efficiency improvements. Overall, this work supports the use of rule-based EMS for HEVs and lays the foundation for future enhancements in hybrid energy management systems.

List of Abbreviations

1. ICE – Internal Combustion Engine
2. EV – Electric Vehicle
3. HEV – Hybrid Electric Vehicle
4. EMS – Energy Management System
5. ML – Machine Learning
6. EM – Electric Motor
7. BESS – Battery Energy Storage System
8. HESS – Hybrid Energy Storage System
9. PHEVs – Plug-in Hybrid Electric Vehicle
10. RL – Reinforcement Learning
11. ECMS – Equivalent Consumption Minimization Strategy
12. MPC – Model Predictive Control
13. ECMS – Equivalent Consumption Minimization Strategy
14. BSFC – Brake-Specific-Fuel-Consumption
15. BESS – Battery Energy Storage System
16. WLTP – Worldwide Harmonized Light Vehicles Test Procedure
17. NEDC – New European Driving Cycle

1. Introduction

1.1. Introduction of the problem

For more than a century, the automotive industry has relied primarily on internal combustion engines (ICEs) as the main source of vehicle propulsion. However, growing environmental concerns, such as climate change and air pollution, along with the global dependence on fossil fuels, have prompted a shift toward cleaner and more sustainable alternatives.¹ In recent years, many governments have announced plans to phase out vehicles powered only by combustion engines due to their harmful emissions and contribution to global warming.²

Although fully electric vehicles (EVs) are seen as the ultimate solution for achieving zero-emission transportation, their widespread adoption remains limited by high battery production costs, limited driving range, and charging infrastructure challenges. As a transitional technology, hybrid electric vehicles (HEVs) offer a practical compromise. By combining an internal combustion engine with an electric motor and battery system, HEVs can improve energy efficiency, reduce emissions, and extend driving range compared to traditional vehicles.

A crucial element in the performance of HEVs is their Energy Management System (EMS) – the control strategy responsible for determining when and how to utilize the electric motor and combustion engine. The EMS directly influences fuel efficiency, energy recovery, and overall vehicle performance. Developing intelligent and adaptive EMS algorithms has therefore become a key area of research in modern automotive engineering.³

This project focuses on the design and simulation of a Smart Powertrain Control System aimed at improving the energy efficiency of HEVs. It investigates how intelligent control algorithms and simulation-based modelling can be used to optimize the interaction between the internal combustion engine and the electric motor. By incorporating modern computational techniques such as machine learning (ML) and data-driven decision-making, the proposed system seeks to achieve an optimal balance between fuel consumption and battery usage. The research also provides insights into current powertrain control strategies and demonstrates how smart simulation tools can contribute to the development of more efficient and environmentally sustainable vehicles.

¹ Panday, A.; Bansal, H. A Review of Optimal Energy Management Strategies for Hybrid Electric Vehicles. *Int. J. Veh. Technol.* 2014, 2014, 160510.

² Morfeldt, J.; Kurland, S.D.; Johansson, D.J. Carbon Footprint Impacts of Banning Cars with Internal Combustion Engines. *Transp. Res. Part D Transp. Environ.* 2021, 95, 102807.

³ M.R.H. Mojumder.; Taher Hasan Nakib , M. Hasanuzzaman , A.S. Abdel-Khalik Advanc., challenges, and prospects of cost-effective EV: A review. 2025,128, 437-456.

<https://www.sciencedirect.com/science/article/pii/S1110016825007173>

1.2. Objective

The primary objective of this project is to design and simulate a Smart Powertrain Control System capable of optimizing the energy efficiency of HEVs. The system aims to smartly manage power distribution between the ICE and the electric motor to minimize fuel consumption and maximize overall performance.

Specific objectives include:

- Develop a simulation model of a hybrid vehicle powertrain that integrates the internal combustion engine, electric motor, and battery system.
- Design and implement an intelligent control algorithm (EMS) to determine optimal power split between the ICE and electric motor under various driving conditions.
- Create a set of realistic driving conditions and load scenarios – such as city traffic, highway cruising, and hill climbing – to evaluate how the EMS dynamically selects the most energy-efficient operating mode.
- Analyse vehicle performance metrics, including fuel consumption, battery usage, and total energy efficiency.
- Visualize and interpret simulation results using data-driven graphs and performance indicators to evaluate system effectiveness.
- Explore the use of intelligent computing techniques, such as rule-based control or machine learning, to enhance decision-making in real time.

1.3. Focus on HEV

Although fully electric vehicles promise higher efficiency, lower running costs, and support for carbon neutrality, they still face major challenges related to battery cost, size, and weight. With the average cost of around 490 zł/kWh the total battery cost reaches nearly 58,800 zł, significantly increasing the vehicle's overall price and making it less accessible to average consumers.⁴

Battery mass also strongly influences energy consumption. The power required for vehicle motion increases with total weight, meaning heavier vehicles consume more energy to travel the same distance. The weight of a Volkswagen Passat about 1635 kg⁵, while a Tesla Model S weighs around 2045 kg⁶ – a 30% increase in mass. This additional weight can lead to an estimated 25% higher energy demand during driving. Unlike conventional cars, which operate on energy produced onboard through an internal combustion engine, EVs rely on external electricity generation and energy storage in batteries. However, both electricity transmission and charging introduce losses – each with around 90% efficiency – resulting in approximately 54% more total energy required from the grid compared to what a conventional vehicle like the Passat would use from fuel. And even if the grid energy is cleaner, these losses reduce the environmental advantage of EVs.

HEVs offer a practical compromise between traditional and fully electric powertrains. They use smaller battery packs alongside downsized engines, minimizing both cost and weight while maintaining efficiency. For example, the Volkswagen Passat GTE Hybrid includes 9.9 kWh battery pack, increasing the total weight by only 65 kg compared to the non-hybrid version. This minor difference keeps the power requirement for motion almost identical, while the hybrid system benefits from regenerative braking and electric propulsion during periods when the engine is less efficient. As a result, fuel consumption improves significantly – from around 7.0 L/100 km (city) and 5.2 L/100 km (highway) to approximately 3 L/100 km (city) and 1.7 L/100 km (highway).⁷

This improvement in fuel efficiency leads to substantial cost savings over time. Although hybrid vehicles may have a higher initial price, lower fuel consumption, government incentives, and decreasing battery costs make them more economical to own in the long run.

⁴ Ziegler, M.S.; Song, J.; Trancik, J.E. Determinants of Lithium-Ion Battery Technology Cost Decline. *Energy Environ. Sci.* 2021, 14, 6074–6098.

<https://pubs.rsc.org/en/content/articlehtml/2021/xx/d1ee01313k>

⁵ 2022 Volkswagen Passat Technical Specifications <https://media.vw.com/assets/documents/original/14662-2022PassatTechnicalSpecsFinalv2.pdf>

⁶ Tesla. Tesla Model S. Available online: <https://www.tesla.com/models>

⁷ Specs of Volkswagen Passat <https://www.auto-data.net/en/volkswagen-passat-b8-gte-1.4-tsi-218hp-plug-in-hybrid-dsg-28003>

Beyond financial savings, the enhanced energy efficiency and reduced fossil fuel consumption of HEVs contribute significantly toward carbon neutrality targets for 2050. Through continuous advancements in smart powertrain control systems and simulation-based energy optimization, HEVs are becoming increasingly effective at balancing performance, cost, and environmental sustainability.⁸

1.4. HEVs Architecture

HEV consists of several core components, including an electric motor (EM), ICE, battery, power converter, fuel tank, and a control unit. These elements are typically grouped into three main systems:

- Drivetrain system – integrates the ICE and electric drive components to transmit power to the wheels.
- Battery Energy Storage System (BESS) – responsible for storing and delivering electrical energy with varying power and capacity levels.
- Control system – manages the interaction between the electric and mechanical subsystems and coordinates the operation of the Hybrid Energy Storage System (HESS).

The combination and configuration of these systems define the overall HEV architecture, leading to different design variations. Based on component integration and power distribution, HEV drivetrains are typically categorized into series, parallel, and power split types.⁹

⁸ Specs of Volkswagen Passat

<https://www.auto-data.net/en/volkswagen-passat-b8-gte-1.4-tsi-218hp-plug-in-hybrid-dsg-28003>

⁹ Abdellatif M. Sadeq Hybrid and Electric Vehicles - A Comprehensive Guide 2024, 10, 1398.

https://www.researchgate.net/publication/379907499_Hybrid_and_Electric_Vehicles_-_A_Comprehensive_Guide

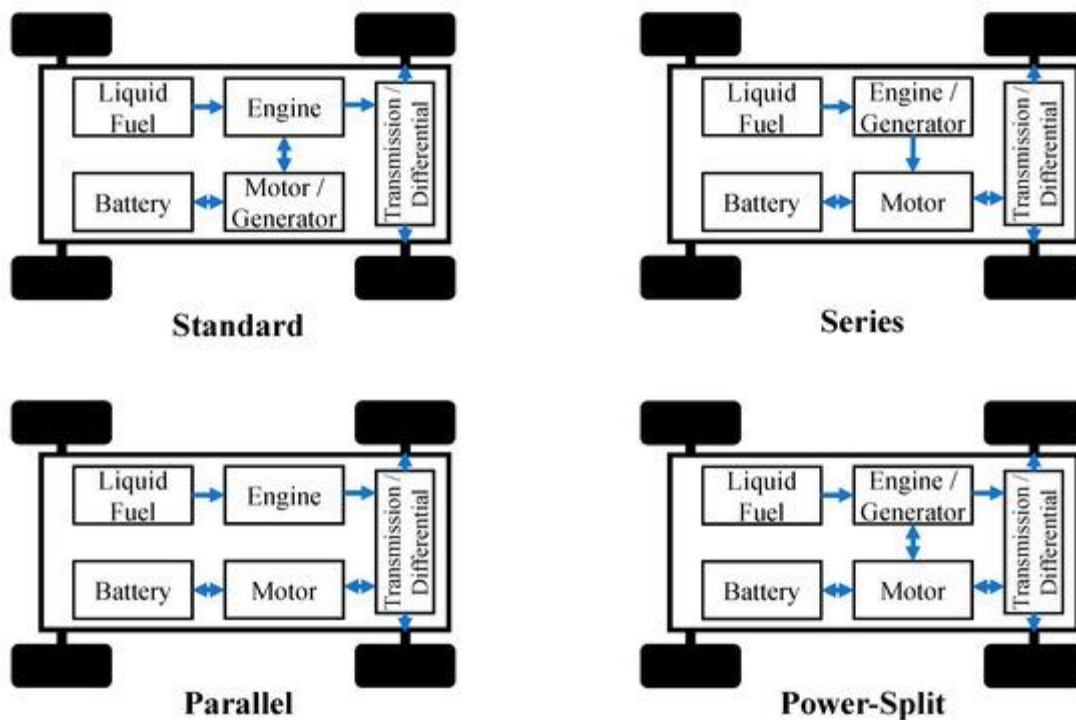


Figure 1. The four different hybrid electric vehicle architectures, created by Vikram Mittal

A series HEV operates similarly to a battery-electric vehicle, with the ICE driving a generator that produces electricity to power the motor or recharge the battery. The motor alone provides traction to the wheels. This setup eliminates the need for a mechanical transmission, reducing overall vehicle weight. Additionally, the engine can run within its optimal efficiency range because it is not directly linked to driving conditions. However, this architecture demands a larger and more expensive battery and generator system, and multiple energy conversions – from mechanical to electrical and back – lead to notable efficiency losses.

In a parallel HEV, propulsion can come from either the ICE or the electric motor, depending on what the EMS determines to be most efficient. At lower speeds, the electric motor typically handles driving duties to maximize fuel economy, while the engine takes over at higher speeds or during heavy loads. Compared to the series configuration, the parallel layout uses a smaller battery pack and minimizes conversion losses since mechanical energy from the engine can be transmitted directly to the wheels.

The power-split hybrid merges the advantages of both series and parallel systems. It incorporates a planetary gear set that connects the engine, motor, and generator, enabling both mechanical and electrical power flows. This design allows the system to dynamically distribute power based on driving conditions, improving fuel efficiency and enhancing overall control of the powertrain.

If to look further, HEVs can be divided into three main categories: mild hybrids, full hybrids, and plug-in hybrids.

Mild hybrids typically use a parallel configuration, where the electric motor assists the engine but cannot power the vehicle independently. The engine shuts off during braking or idling to improve fuel economy, yet propulsion relies entirely on the ICE.

Full hybrids and plug-in hybrids (PHEVs), on the other hand, generally employ power-split architectures, allowing the electric motor to drive the vehicle either alone or in combination with the engine. Both types recharge their batteries using an onboard generator during driving, while plug-in hybrids have the added ability to charge directly from an external electrical source.^{10 11}

This project concentrates only on full and plug-in hybrid vehicles, as these systems are considerably more sophisticated than those used in mild hybrid configurations

1.5. Role of EMS

In power-split hybrid electric vehicles, the EMS is a key control layer that determines how engine and electric motor power are combined to meet the driver's demand while reducing overall fuel use and emissions. To make these decisions, the EMS continuously receives information from the vehicle – such as speed, SOC, and accelerator input – and processes these signals through a control algorithm that allocates power between the internal combustion engine and the electric machine. By regulating this power distribution, the EMS ensures that the battery maintains an appropriate SOC level and that the powertrain operates as efficiently as possible.

Over time, EMS strategies have evolved from basic rule-based approaches to more advanced algorithms capable of improving hybrid performance by anticipating energy needs and optimizing power flow under a wider range of driving conditions. As EMS sophistication increases, so does the potential for improved fuel economy and reduced emissions.¹²

¹⁰ Ripaccioli, G.; Bernardini, D.; Di Cairano, S.; Bemporad, A.; Kolmanovsky, I.V. A Stochastic Model Predictive Control Approach for Series Hybrid Electric Vehicle Power Management. In Proceedings of the 2010 American Control Conference, Baltimore, MD, USA, 30 June 2010; pp. 5844–5849.

<https://ieeexplore.ieee.org/abstract/document/5530504>

¹¹ Zhang, F.; Wang, L.; Coskun, S.; Pang, H.; Cui, Y.; Xi, J. Energy Management Strategies for Hybrid Electric Vehicles: Review, Classification, Comparison, and Outlook. *Energies* **2020**, *13*, 3352.

<https://www.mdpi.com/1996-1073/13/13/3352>

¹² Zhang, F.; Wang, L.; Coskun, S.; Pang, H.; Cui, Y.; Xi, J. Energy Management Strategies for Hybrid Electric Vehicles: Review, Classification, Comparison, and Outlook. *Energies* **2020**, *13*, 3352.

<https://www.mdpi.com/1996-1073/13/13/3352>

1.6. Effect of EMS

The EMS has a decisive impact on the overall fuel efficiency of a hybrid vehicle. If the control strategy relies excessively on the internal combustion engine, the additional weight of the hybrid components may cause the vehicle to consume more fuel than a comparable non-hybrid model. Conversely, when the EMS appropriately prioritizes electric operation and restricts engine use to conditions where it is most efficient, hybrid vehicles can achieve excellent energy performance.

To evaluate this performance in metric units, energy efficiency is often expressed as kilometres per liter equivalent (km/L_e), which indicates the distance a vehicle can travel using the amount of energy contained in one liter of gasoline. It can be calculated as:¹³

$$\text{km/L}_e = \frac{\text{Distance traveled (km)}}{\text{Energy consumed (kWh)}} \times \text{Energy content of gasoline (kWh/L)}$$

Figure 2. Kilometres-per-Liter equation, created in Microsoft Word by Illia Novikov.

A well-implemented EMS can allow an HEV to reach km/L_e values comparable to, or in some cases better than, those of certain electric vehicles. This is partly due to the smaller battery typically used in hybrids, which reduces total vehicle mass and therefore lowers the propulsion energy required. However, designing an optimal EMS remains a major challenge. The system must simultaneously respond to the driver's immediate power demands and maintain long-term fuel efficiency, all while operating under unpredictable and rapidly changing driving conditions. Ideally, the EMS would accurately anticipate upcoming power requirements and adjust the engine-motor power split accordingly, but predicting driver behaviour, road load variations, and battery dynamics introduces significant complexity. As a result, developing an EMS that consistently improves efficiency without sacrificing performance continues to be a demanding engineering problem.¹⁴

¹³ Jung, H. Fuel Economy of Plug-In Hybrid Electric and Hybrid Electric Vehicles: Effects of Vehicle Weight, Hybridization Ratio and Ambient Temperature. *World Electr. Veh. J.* 2020, 11, 31.
<https://www.mdpi.com/2032-6653/11/2/31>

¹⁴ Jalil, N.; Kheir, N.A.; Salman, M. A Rule-Based Energy Management Strategy for a Series Hybrid Vehicle. In Proceedings of the 1997 American Control Conference, Albuquerque, NM, USA, 6 June 1997; pp. 689–693.
<https://ieeexplore.ieee.org/abstract/document/611889>

2. Aim of Project

The aim of this project is to design and simulate an intelligent powertrain control system for HEVs that optimizes energy efficiency under various driving conditions. The study focuses on evaluating how modern control strategies, predictive algorithms can improve overall HEV performance and extend their relevance in the automotive market. Additionally, the project analyses how far these improvements can compete with the rapidly advancing fully EV technologies, particularly in terms of efficiency, battery cost, and practical usability, to assess the future role of HEVs in the transition toward full electrification.

2.1. Insights and Prospects

Modern hybrid energy management largely relies on rule-based and optimization-based control strategies that interpret real-time information such as battery SOC, vehicle speed, and instantaneous power demand. Although these methods are reliable and computationally efficient, they are inherently reactive: they adjust to the vehicle's current operating state without considering what will happen ahead. As a result, many factors influencing energy efficiency remain outside the scope of today's EMS implementations.

Future energy needs, driving environment transitions, and external contextual information – such as upcoming traffic patterns, road topology, weather conditions, or driver-specific habits – are not captured by conventional EMS frameworks. These omitted variables can significantly affect propulsion efficiency. For instance, a system unaware of future road grade changes or anticipated braking opportunities may allocate power not fully optimal, reducing the overall energy performance of the hybrid powertrain.¹⁵ Looking forward, the EMS is expected to progress toward predictive and context-aware architectures. Advancements in machine learning, vehicle connectivity, cloud computing, and perception systems will gradually transform hybrid power management from reaction-driven control to forecast-based decision-making. Reinforcement learning (RL) approaches already demonstrate the ability to achieve optimal control policies by learning from past driving behaviour and operational data. These algorithms adapt to diverse driving patterns and show superior fuel economy compared with classical optimization techniques.¹⁶

¹⁵ Torreglosa, J.P.; Garcia-Triviño, P.; Vera, D.; López-García, D.A. Analyzing the Improvements of Energy Management Systems for Hybrid Electric Vehicles Using a Systematic Literature Review: How Far Are These Controls from Rule-Based Controls Used in Commercial Vehicles? *Appl. Sci.* 2020, *10*, 8744.

¹⁶ Jane, R.S.; James, C.; Kim, T. Using AI-ML to Develop Energy and Exergy Flow Characterizations for Multi-Domain Operations. In Proceedings of the Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications, Baltimore, MD, USA, 12 April 2021; Volume 11746, p. 1174628. <https://www.spiedigitallibrary.org/conference-proceedings-of-spie/11746/1174628/Using-AI-ML-to-develop-energy-and-exergy-flow-characterizations/10.1117/12.2585775.short>

Over the mid-term to long-term horizon, EMS computation will increasingly shift away from the vehicle into cloud-based processing environments. With access to a broader information ecosystem – traffic networks, infrastructure data, environmental updates, and even behavioural data from surrounding vehicles – the EMS will be capable of significantly more informed decision-making. As vehicles become more connected, cooperative energy management strategies leveraging information from close vehicles and infrastructure will continue improving predictive accuracy.

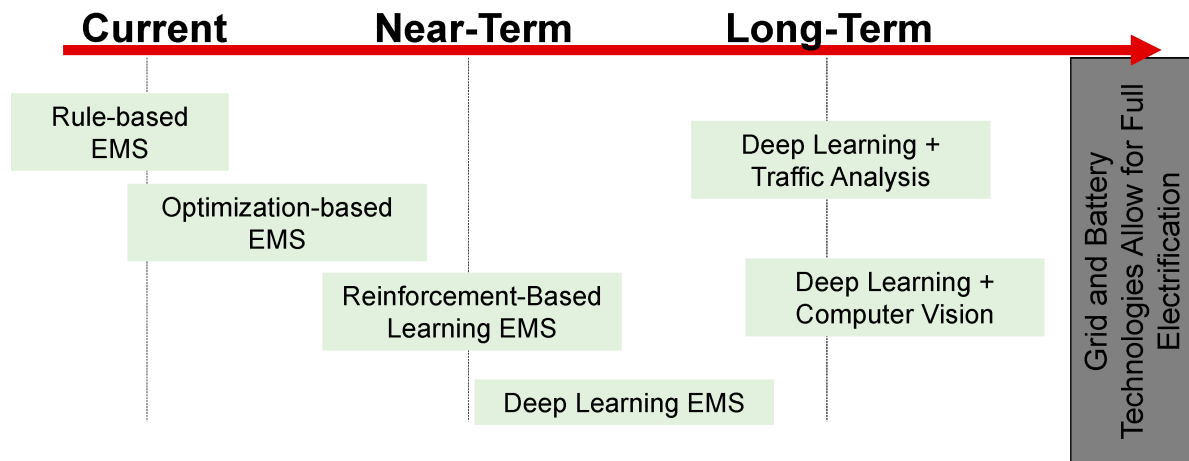


Figure 3. Technology roadmap for the evolution of EMSs for HEVs by World Electric Vehicle Journal (WEVJ).

Considering the rapid pace of electrification, it is possible that hybrid vehicles will gradually decline as battery chemistries evolve and fully electric vehicles become lighter, cheaper, and more efficient. However, the evolution of EMS technologies remains essential, as these innovations provide a bridge to future mobility systems and support the transition toward smarter, more adaptive propulsion control.¹⁷

2.2. Issues and Solutions

Despite substantial progress, existing EMS approaches face several limitations that restrict their ability to operate at peak efficiency. The primary issue is the absence of predictive capability. Conventional control strategies do not account for forthcoming power demands or environmental conditions, leaving the system unable to prepare for events such as extended deceleration, sudden idling, significant elevation changes, or prolonged low-speed driving. This lack of foresight results in suboptimal battery usage, inefficient engine operation, and missed opportunities for regenerative energy capture. Another challenge lies in computational constraints. Predictive models, machine

¹⁷ Jane, R.S.; James, C.; Kim, T. Using AI-ML to Develop Energy and Exergy Flow Characterizations for Multi-Domain Operations. In Proceedings of the Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications, Baltimore, MD, USA, 12 April 2021; Volume 11746, p. 1174628. <https://www.spiedigitallibrary.org/conference-proceedings-of-spie/11746/1174628/Using-AI-ML-to-develop-energy-and-exergy-flow-characterizations/10.1117/12.2585775.short>

learning algorithms, and large-scale data fusion methods often require processing capabilities that exceed the capacity of embedded automotive controllers. This restricts the adoption of more advanced algorithms, especially in lower-cost vehicle platforms.

Several solutions exist to address these challenges. First, reinforcement learning and other adaptive algorithms provide a path toward EMSs that can learn from data and adjust control strategies without the need for extensive preprogrammed rules. These models can better handle unpredictable driving conditions and adapt to long-term patterns in driver behaviour. Second, cloud off-loading offers a practical way to incorporate computationally demanding algorithms while maintaining feasible hardware requirements inside the vehicle. Third, integration of vehicle-to-vehicle and vehicle-to-infrastructure communication can supply the EMS with valuable external information that enhances predictive decision-making. Finally, advancements in perception – particularly through computer vision – allow the EMS to interpret the driving environment more accurately, enabling finer-grained optimization decisions.¹⁸

3. Literature Overview

3.1. HEV Powertrain Architecture

3.1.1. ICE

The ICE remains a central component of HEVs, providing high energy density and sustained power delivery across a wide range of operating conditions. In the context of powertrain simulation, the ICE is typically represented through a quasi-static model that relates engine speed and torque to fuel consumption and efficiency. This model is based on experimentally derived fuel maps, which quantify how much fuel is required to produce a specific torque at a given rotational speed. The torque limits of the ICE – such as minimum stable torque, peak torque curve, and idle speed – define the feasible operating range within which the engine can safely and efficiently function.

Another important theoretical aspect is the concept of brake-specific fuel consumption (BSFC), which indicates how effectively the engine converts fuel energy into mechanical output. The EMS relies on these maps to determine when the engine should operate to maximize efficiency, or alternatively, when it should remain off to prevent unnecessary fuel usage. Modern engines also exhibit highly nonlinear characteristics due to intake airflow dynamics, friction losses, and throttle behaviour, but for hybrid simulation, these complexities are often reduced to algebraic relationships linking torque, speed, and fuel rate.¹⁹

¹⁸ Li, D.; Zhao, D.; Zhang, Q.; Chen, Y. Reinforcement Learning and Deep Learning Based Lateral Control for Autonomous Driving [Application Notes]. *IEEE Comput. Intell. Mag.* 2019, 14, 83–98. <https://ieeexplore.ieee.org/abstract/document/8686348>

¹⁹ Brake Specific Fuel Consumption (BSFC): <https://x-engineer.org/brake-specific-fuel-consumption-bsfc/>

In a hybrid drivetrain, the ICE does not always respond directly to the driver's torque demand. Instead, the EMS determines whether the engine should provide power to the wheels, recharge the battery, or remain inactive. This decoupling between driver demand and engine operation is a critical aspect of HEV modelling: energy management, rather than instantaneous power follow, dictates engine usage. Thus, including an accurate ICE model is essential for predicting realistic fuel consumption and evaluating how the EMS balances energy flows between the engine and electric components.

3.1.2. Electric Machine (Motor/Generator)

The electric machine in HEV functions both as a traction motor and a generator, depending on the operational mode dictated by the EMS. As a traction motor, it supplements or replaces the engine by delivering torque directly to the drivetrain. During regenerative braking or load management, it operates as a generator, converting mechanical energy back into electrical energy stored in the battery. In simulations, the electric machine is usually modelled through efficiency or loss maps that relate torque and speed to electrical power consumption or generation. These maps account for metal losses, inverter losses, and mechanical friction, providing a realistic representation of machine performance across its operating envelope.²⁰

The electric machine's ability to deliver torque instantaneously and with high efficiency makes it a central element in hybrid operation. Unlike the ICE, which has slower dynamic response and optimal efficiency within a narrow diapason, the electric machine can efficiently deliver torque across a wide range of speeds. The EMS exploits this characteristic to smooth power demand, reduce fuel consumption, and maintain the battery's SOC.

In generator mode, the motor's torque is negative, absorbing mechanical energy and converting it into electrical energy through controlled rectification and inverter modulation. This bidirectional capability requires modelling both propulsion and regeneration limits, which are constrained by inverter ratings, thermal limits, and battery charging power.

In powertrain, the electric machine provides the flexibility needed to redistribute load between the engine and battery. The EMS determines the contribution of the motor based on instantaneous demand, battery SOC limits, and predefined operational rules. A detailed motor model therefore ensures realistic predictions of system efficiency, regenerative braking potential, and electrical energy usage.

²⁰ Electric Vehicle Motor: <https://x-engineer.org/electric-vehicle-motor-torque-power-curves/>

3.1.3. Battery Energy Storage System (BESS)

The BESS is responsible for storing electrical energy and supplying power to the electric machine. In hybrid simulation, the battery is typically modelled using an equivalent circuit approach, most commonly with an open-circuit voltage (OCV) and an internal resistance that varies with SOC. This representation captures the nonlinear voltage-current relationship and the energy losses associated with charging and discharging. The SOC is updated at each timestep according to the current drawn or supplied, ensuring the simulation maintains realistic behaviour regarding capacity limits and efficiency.²¹

One of the most important theoretical concepts is that the battery cannot freely charge or discharge beyond its allowable power limits. Maximum charging power is restricted by internal chemistry and thermal considerations, while maximum discharge power is limited by voltage drop and current handling capability. These constraints directly influence the EMS's decisions: if SOC falls below a lower threshold, the EMS will reduce motor assist and prioritize engine operation or charging; if SOC approaches an upper threshold, regenerative braking must be limited or curtailed to prevent overcharging.²²

Battery efficiency also plays a crucial role. Charging and discharging efficiencies are less than 100%, meaning some energy is always lost as heat. The simulation typically incorporates Coulombic efficiency or separate charge/discharge factors. Additionally, temperature and aging effects impact battery performance and available energy. Although this is in more complex simulations.

In the powertrain model, the battery interacts dynamically with the motor and engine. The EMS acts as the agent, determining when to draw power, when to charge, and how to maintain SOC balance over the driving cycle. A well-constructed battery model is fundamental for obtaining realistic results in HEV simulations, as SOC constraints heavily influence feasible power splits and long-term energy sustainability.

3.1.4. Vehicle Longitudinal Dynamics

Vehicle longitudinal dynamics describe how the vehicle responds to applied tractive force and resistive loads during motion. These dynamics are represented through a set of physical equations relating wheel torque to vehicle acceleration, considering forces such as aerodynamic drag, rolling resistance, gravitational load on inclines, and

²¹ What is a battery electric vehicle: <https://x-engineer.org/battery-electric-vehicle-bev/>

²²Anna Tomaszewska, Zhengyu Chu, Xuning Feng , Simon O'Kane, Xinhua Liu, Jingyi Chen, Chenzhen Ji, Elizabeth Endler, Ruihe Li, Lishuo Liu, Yalun Li , Siqi Zheng , Sebastian Vetterlein , Ming Gao , Jiuyu Du , Michael Parkes, Minggao Ouyang, Monica Marinescu, Gregory Offer, Billy Wu Lithium-ion battery fast charging: A review 2019, 1, 100011. <https://www.sciencedirect.com/science/article/pii/S2590116819300116>

inertial effects. To accurately model the powertrain all those factors should be taken into account.

Aerodynamic drag increases with the square of vehicle speed and is defined by parameters such as drag coefficient, frontal area, and air density. Rolling resistance is modelled as a constant force proportional to vehicle weight, while gravitational force plays a role when the vehicle travels on slopes. Together, these forces dictate the power required at the wheels to follow the target speed profile.²³

The longitudinal dynamics model therefore acts as the “environment” that the powertrain must satisfy. For any given timestep, the simulation computes the required traction force to meet the speed demand. This requirement is then translated into torque that must be provided by the engine, motor, or both. The EMS uses the available torque capacities and system constraints to decide how that demand will be distributed.

Accurate longitudinal modelling ensures that the simulation captures realistic transient behaviour, such as acceleration, deceleration, and steady-state cruising. Without it, the powertrain would not be subject to real-world driving loads, and the resulting fuel or energy consumption predictions would be meaningless. As such, vehicle dynamics form the backbone of the HEV simulation, linking energy management decisions to physical vehicle motion.

3.1.5. Controller (EMS)

The Controller is the central decision-making unit that coordinates power distribution between the engine, electric machine, and battery. In rule-based strategies, the EMS relies on predefined logic derived from engineering insight, vehicle behaviour analysis, and component efficiency characteristics. Instead of optimizing over the entire drive cycle, it reacts instantaneously to the current vehicle state, ensuring that each component operates within its limits while maintaining overall system balance.²⁴

The EMS receives inputs such as demanded wheel power, vehicle speed, engine and motor limits, and battery SOC. Using these inputs, it selects an operational mode: engine-only, motor-only, hybrid assist, regenerative braking, or charging. The core objective is to keep the battery within an acceptable SOC range while ensuring the vehicle meets driver demand and minimizing fuel usage. Rules often include thresholds

²³ Tire model for longitudinal forces: <https://x-engineer.org/tire-model-longitudinal-forces/>

²⁴ Ed. Louback, A. Biswas, Fab. Machado, Ali Emadi A review of the design process of energy management systems for dual-motor battery electric vehicles 2024, 193, 114293.
<https://www.sciencedirect.com/science/article/pii/S1364032124000169>

for engine activation, torque split boundaries, SOC recovery strategies, and constraints to prevent overloading components.

A key theoretical concept is that the EMS must satisfy power demand while considering efficiencies of both the engine and electric machine. For example, when demand is low, the EMS may choose motor-only operation to avoid inefficient engine regions. At moderate loads, it may operate the engine within its efficient range and supplement the rest with electric power. During braking, it maximizes regeneration without violating battery charging limits.²⁵

In simulation, the EMS serves as the link between all subsystem models. The behaviour of the engine, motor, battery, and vehicle dynamics is ultimately shaped by the EMS rules. The accuracy of the EMS is essential for producing realistic fuel consumption results and understanding how energy flows through the hybrid powertrain under varying driving conditions.

3.2. HEV Control Strategies Overview

3.2.1. Rule-based

Rule-based EMS represent the foundational control approach used in hybrid electric vehicles and remain one of the most widely adopted methods in both research and industrial applications. A rule-based EMS control the hybrid powertrain by specifying a set of operating rules that determine how the engine, electric motor, and battery interact under different driving conditions. These rules are typically derived from engineering expertise, heuristic observations, and simplified physical models, and they are formulated to reflect how a hybrid system should behave in real-world scenarios. The purpose of these rules is to maintain the powertrain within favourable operating regions while ensuring that both the ICE and battery support the overall energy efficiency and driving performance of the vehicle.²⁶ Rule-based control approaches can generally be grouped into two major categories: deterministic and fuzzy-logic-based strategies.

Deterministic rule-based strategies rely on explicitly defined conditions and thresholds. These include, for example, SOC thresholds for switching between electric-only, hybrid, or charging modes; torque split logic based on driver demand; or engine activation rules tied to vehicle speed or power requirements. Deterministic controllers usually reflect a combination of empirical data, engineering judgement, and component

²⁵ Ah. K. Abbas, Raz. Ayop Advanced energy-management and sizing techniques for renewable microgrids with electric-vehicle integration: A review 2025, 27, 106252. <https://www.sciencedirect.com/science/article/pii/S2590123025023242>

²⁶ Theo Hofman, Alex Serrarens A Rule-based energy management strategies for hybrid vehicles 2007, 150, 014448. https://www.researchgate.net/publication/228679078_A_Rule-based_energy_management_strategies_for_hybrid_vehicles

limitations. They respond directly to driving demands and system states, ensuring that each subsystem stays within its allowed operating range while delivering the required vehicle performance. This type of EMS is widely used in production hybrids from manufacturers such as Toyota and Honda, where predictable, stable behaviour across diverse operating scenarios is essential.

Fuzzy-logic-based rule strategies, in contrast, do not rely solely on strict thresholds. Instead, they incorporate fuzzy sets and membership functions to interpret variables such as driver power demand, SOC, or speed in a gradual, continuous manner. This introduces smoother transitions and allows the controller to account not only for the magnitude of a variable but also for the rate at which it changes. Such controllers can manage uncertain states more effectively than deterministic rules, especially during situations where driving behaviour fluctuates rapidly. By adjusting rules based on fuzzy logic provides a more flexible decision-making process while still remaining within the framework of rule-based control.²⁷

Although rule-based strategies are sometimes viewed as less adaptive than predictive or optimization-based EMS architectures, their importance is substantial. They remain essential in hybrid vehicle research and are still central to many commercial HEVs. Their behaviour is quite understandable, their performance is repeatable across different driving cycles, and their structure allows engineers to embed domain knowledge directly into the control logic. These advantages make rule-based strategies particularly suitable for systems where robustness, reliability, and transparency are critical. However, their performance may vary with driving cycle characteristics, trip length, traffic variability, and environmental conditions.

Nevertheless, rule-based EMSs remain a cornerstone in HEV control research. They serve as a benchmark against which more advanced controllers – such as model predictive control, Equivalent Consumption Minimization Strategy (ECMS), or reinforcement-learning-based EMS – are evaluated. Their continued use in academic simulation studies highlights the method's reliability, traceability, and practical usage. Because the rules can be explicitly defined, modified, and validated, this approach is especially valuable for projects focused on understanding hybrid powertrain behaviour, analysing component interaction, and assessing the impact of control decisions on fuel consumption and SOC evolution, and HEVs future overall.

²⁷ Farzad Raj. Salmasi Control Strategies for Hybrid Electric Vehicles: Evolution, Classification, Comparison, and Future Trends 2007, 109, 899933. https://www.researchgate.net/publication/271526695_Control_Strategies_for_Hybrid_Electric_Vehicles_Evolution_Classification_Comparison_and_Future_Trends

3.2.2. Optimization-based

Optimization-based EMS aim to minimize a mathematically defined cost function that represents the vehicle's operational goals. This cost function typically combines fuel consumption, emissions, battery usage, and performance-related requirements such as delivered torque or drivability. In contrast to rule-based EMS, which directly dictates how components should behave at each moment, optimization-based approaches derive their control logics from solving a formal optimization problem. The resulting control signals are not fixed heuristics but are computed to achieve the lowest possible cost for a given set of conditions. A central aspect of optimization-based strategies is that they often rely on drive-cycle information. These methods compute the optimal split between mechanical and electrical power for an entire sequence of future vehicle speeds. As a result, the solutions they provide are cycle-specific – meaning they are optimal only for the speed profile used in the computation. Without knowing the future driving demand, these control laws cannot be directly deployed in real-time vehicle operation. Therefore, the effectiveness of these strategies is strongly tied to the availability and accuracy of predictive driving information.²⁸

Optimization-based EMSs are generally divided into two major categories: global optimization and instantaneous optimization.

Global optimization seeks a control protocol that minimizes the total cost over the entire driving cycle. Techniques such as DP, genetic algorithms, or other nonlinear programming methods compute the best possible engine-motor power split over the full trajectory of vehicle speeds. These strategies require complete knowledge of future events like:

- vehicle speed and acceleration
- driver power demand
- road grade or traffic patterns
- battery SOC evolution

Global optimization therefore sets a theoretical upper bound on what EMS can achieve. It provides a benchmark for evaluating other control strategies and is widely used in academic/research industry. But because techniques like DP solve optimization problems with a lot of stages, the computational requirements scale dramatically with the number of states and variables, as exhaustive search or recursive cost computation happens. Even with modern computing, this makes global optimization

²⁸ Vaishnavi Tella, Mohamad Alzayed A Comprehensive Review of Energy Management Strategies in Hybrid Electric Vehicles: Comparative Analysis and Challenges 2024, 109, 509737. https://www.researchgate.net/publication/386360232_A_Comprehensive_Review_of_Energy_Management_Strategies_in_Hybrid_Electric_Vehicles_Comparative_Analysis_and_Challenges

unsuitable for real-time deployment in production HEVs.

Instantaneous optimization addresses the limitations of global optimization by minimizing cost at each individual time step rather than over the entire cycle. Instead of requiring full knowledge of the future, these controllers use real-time measurements and short-term predictions to determine the optimal power distribution. Two dominant instantaneous strategies are:

- Model Predictive Control (MPC)
- Equivalent Consumption Minimization Strategy (ECMS)

MPC balances optimality and real-time applicability. It does not require full knowledge of the entire drive cycle but still benefits from short-horizon preview information. This makes it an attractive strategy for modern HEVs, especially in cases where limited prediction from navigation systems or traffic data is available. ECMS simplifies the complex multi-stage optimization problem by converting instantaneous electrical energy use into an equivalent fuel consumption using a weighting factor (the “equivalence factor”). This turns the overall cost function into a per-time-step minimization problem.²⁹

By minimizing total equivalent fuel at each instant, ECMS approximates the optimal global control policy without requiring knowledge of future states. A properly calibrated ECMS can achieve performance close to global DP while remaining computationally feasible for real-time implementation.

3.2.3. Dynamic Rule-based

Between pure rule-based and full optimization strategies lies a category – dynamic rule-based EMS. These controllers maintain a rule-based structure but continuously adjust the rule parameters based on driving conditions, battery state, or predicted demand.

Researches demonstrated that embedding optimization principles into a rule-based framework can achieve near-optimal results while remaining suitable for real-time execution. Their findings indicated that, for trips up to 120 km, using elements of dynamic programming in a rule-based approach reduced fuel consumption by around 15% compared to a traditional rule-based EMS. Such reductions translate to meaningful efficiency gains across the entire lifetime of the vehicle.³⁰

²⁹ H. Tarout., H. Zaki, A. Chahbouni, E. Ennajih, El M. Louragli Optimizing Energy Consumption in Electric Vehicles: A Systematic and Bibliometric Review of Recent Advances 2025, 577, 100577. <https://www.mdpi.com/2032-6653/16/10/577>

³⁰ Vispi Karkaria, Ying-Kuan Tsai An optimization-centric review on integrating artificial intelligence and digital twin technologies in manufacturing 2025, 161, 243420. <https://www.tandfonline.com/doi/full/10.1080/0305215X.2024.2434201>

This hybrid approach effectively combines the robustness and interpretability of rule-based EMS with the performance advantages of optimization, offering a promising direction for HEV control design.

3.3. Driving Cycles and Data Used

Driving cycles play a fundamental role in HEV analysis because they define the time-varying driving conditions under which powertrain components must perform. A driving cycle is typically represented as a velocity-time profile that captures representative patterns of real-world driving, including acceleration, deceleration, cruising, and idling periods. These cycles provide a controlled and repeatable framework for evaluating vehicle fuel consumption, battery behaviour, and EMS.

Standardized regulatory cycles, such as the Worldwide Harmonized Light Vehicles Test Procedure (WLTP), the New European Driving Cycle (NEDC), and the US-based FTP-75, were developed to ensure consistent comparisons across vehicle technologies. WLTP reflects aggressive accelerations and higher average speeds, making it more suitable for evaluating modern HEVs whose control systems respond strongly to transient loads. Meanwhile, FTP-75 focuses on urban stop-and-go driving, which challenges regenerative braking and low-speed assistance, making it valuable for analysing hybridization benefits. The choice of cycle significantly influences the resulting fuel economy, SOC trajectory, and the stress imposed on the battery.³¹ In simulation environments, the driving cycle is used as the primary input to the vehicle longitudinal dynamics model. At each timestep, vehicle speed is differentiated to compute acceleration, which in turn determines the demanded traction force. This required power becomes the main input for the EMS, guiding decisions regarding how the engine, electric machine, and battery should share the load.

Using driving cycles also ensures that the EMS is tested over a wide range of realistic operating points, including high-load events, regenerative braking phases, and idle periods. The quality of EMS performance cannot be evaluated meaningfully without an appropriate driving cycle to stimulate the system.

3.4. Modelling Approaches

HEV powertrain simulation relies on mathematical models that satisfy physics with

³¹ Elia Grano, Manfredi Villani, Manfredi Villani Are We Testing Vehicles the Right Way? Challenges of Electrified and Connected Vehicles for Standard Drive Cycles and On-Road Testing 2025, 94, 020094.
<https://www.mdpi.com/2032-6653/16/2/94>

computational efficiency. In research settings, models are typically simplified to quasi-static or low-order formulations, enabling fast evaluation of energy management strategies without requiring highly detailed component dynamics. Quasi-static models are widely accepted for EMS development and preliminary design studies.

A common modelling framework distinguishes between forward-facing and backward-facing approaches. Backward-facing models compute the required power at each timestep directly from the driving cycle and propagate energy flows backward through the drivetrain by applying component efficiencies. This approach is computationally efficient but unsuited for real-time control testing. In contrast, forward-facing models, such as the one used in this research, derive wheel forces from longitudinal dynamics and simulate the behaviour of each component step-by-step. This structure allows the EMS to affect instantaneous torque allocations, engine or motor decisions, and SOC evolution in a realistic closed-loop manner.

Component models are generally built using experimentally derived static maps. The internal combustion engine is represented by a BSFC surface, which provides fuel flow as a function of torque and speed. Electric machines rely on efficiency maps that capture losses from copper resistance, core effects, and inverter operation, without explicitly modelling electromagnetic fields. Batteries are often described using equivalent circuit models, where the open-circuit voltage and internal resistance depend on SOC. This level of simplification captures most essential electrical behaviours.

Vehicle dynamics models also adopt simplified formulations, combining aerodynamic drag, rolling resistance, gravitational gradients, and inertial terms into a single force balance equation. These assumptions sufficiently reproduce realistic load conditions while keeping the computation light enough for iterative EMS testing.

Overall, the modelling approach used in HEV research aims to reproduce the dominant energy flows and constraints while showing fast-scale dynamics that do not significantly affect control strategies. This balance of accuracy and simplicity makes quasi-static and reduced-order models the standard choice for EMS simulation studies.

4. Application of Project

4.1. Planned Work

The planned work for this project focuses on developing, integrating, and validating a simulation framework that models the energy flows and power management strategies of a HEV. The primary purpose is to create a computational environment where different powertrain components – ICE, electric machine, and BESS – interact through a supervisory EMS. This framework enables controlled experiments on fuel consumption, energy efficiency, and component load distribution under standardized driving cycles.

First, the project aims to finalize and verify all subsystem models. These include an approximate map-based ICE model, a simplified torque-speed electric machine model, a lumped-parameter battery model, and a quasi-static longitudinal vehicle dynamics model. Each component is defined with a balance between computational efficiency and physical accuracy, ensuring that the simulation can run iteratively over long drive cycles such as WLTP or FTP-75 while still reflecting realistic power and energy trends.

Second, the project will implement a rule-based EMS responsible for power split decisions between the engine and electric motor. The EMS will communicate with all subsystems using real-time inputs such as demanded wheel torque, SOC, engine torque limits, and motor efficiency regions. Its objective is to minimize instantaneous fuel consumption while keeping the battery SOC within safe boundaries. The EMS logic will be modular to allow future upgrades, such as replacing the rule-based approach with an optimization-based / machine-learning-based controller.

Third, the planned work includes running simulations on various drive cycles (WLTP-like, FTP-75, and custom cycles). The outputs – fuel consumption, SOC evolution, engine operating points, and motor torque contribution – will be collected and visualized. These results will allow assessment of how component interactions influence energy usage across different driving conditions.

Finally, the project aims to provide a structured basis for future extension. The simulation platform can be adapted to examine alternative EMS strategies. The long-term purpose is to build a flexible, transparent environment for HEV powertrain analysis, educational demonstrations, and early-stage control strategy prototyping for understanding the whole HEV future prospects and supporting further research into more sustainable, efficient, and intelligent vehicle propulsion systems.

4.2. Outcomes

The expected outcomes of this project content both technical deliverables and broader educational value. First, the project is expected to produce a fully functional and modular HEV powertrain simulation capable of representing the key subsystems. The simulation should generate realistic energy-flow and power-usage profiles, including engine operating points, motor torque and electrical power trajectories, battery SOC evolution, and total traction power demands. A major outcome is a detailed evaluation of the rule-based EMS, allowing the project to quantify fuel consumption, examine SOC sustainability, and analyse how component operating points respond to control decisions. The platform should also enable direct comparison between EV-only, ICE-only, and hybrid operating modes by selectively enabling or disabling specific components, providing insight into how hybridization improves overall efficiency.

Additionally, the project is expected to deliver clear analytical outputs such as CSV logs, summary tables, and plots illustrating speed tracking, SOC behaviour, power distribution, and fuel consumption. These tools will support structured analysis and make the system suitable for demonstration and reporting. Another important outcome is the establishment of a flexible foundation for future research, where the simulation framework can be extended to incorporate more advanced EMS strategies. Finally, the project aims to serve as an educational platform, helping future students and researchers understand subsystem interactions, the influence of EMS decisions on energy flow, and the role of different driving cycles in determining vehicle performance. Overall, the project is expected to produce a comprehensive, transparent, and extensible environment that supports both technical analysis and conceptual understanding of HEV energy management.

4.3. Practical Applications

4.3.1. Models Parameters

The simulation framework developed in this project relies on parameter values derived from the 2020 Toyota Prius Hybrid (XW50, 4th generation facelift). This vehicle was selected because it is one of the most extensively studied and benchmarked HEVs in both academic literature and industry testing. Its well-documented specifications make it an ideal reference vehicle for testing hybrid powertrain control strategies and for ensuring that the simulation reflects realistic operational behaviour.

```
#1-D longitudinal vehicle model (Prius).
#Units: meters, seconds, kg, W

def __init__(self,
              mass_kg=1410.0,
              Cd=0.24,
              A_m2=2.34,
              Cr=0.0095,
              rho_air=1.225,
              driveline_eff=0.92):
    self.m = mass_kg
    self.Cd = Cd
    self.A = A_m2
    self.Cr = Cr
    self.rho = rho_air
    self.g = 9.81
    self.driveline_eff = driveline_eff
```

Figure 4. Source: Screen capture of 'hev_project' './src/models/vehicle.py' from VS Code, created by Illia Novikov.

The following parameters were adopted to accurately reflect the physical behaviour of the Prius in longitudinal motion:

- **Vehicle Mass (1410 kg):**
Represents the curb weight of the Prius including standard equipment but excluding passengers or cargo. This mass is consistent with published Toyota specifications for the hybrid trim with a nickel-metal hydride (NiMH) battery.
- **Aerodynamic Drag Coefficient ($C_d = 0.24$):**
The Prius is known for its highly optimized aerodynamic body. A drag coefficient of 0.24 places it among the most aerodynamically efficient production cars, reducing highway fuel consumption and enabling realistic modelling of aerodynamic drag forces at higher speeds.
- **Frontal Area ($A = 2.34 \text{ m}^2$):**
Derived from the width and height of the vehicle, the frontal area is necessary for computing aerodynamic drag. This value is commonly used in academic modelling of the Prius and aligns with measured approximations.
- **Rolling Resistance Coefficient ($C_r = 0.0095$):**
Represents the energy losses due to tire deformation and road interaction. The Prius uses low-rolling-resistance (LRR) tires, typically yielding coefficients between 0.009 and 0.010. A value of 0.0095 fits within this range and provides an accurate representation of rolling friction.
- **Air Density ($\rho = 1.225 \text{ kg/m}^3$):**
Standard sea-level air density under typical environmental conditions. This value is widely used in vehicle dynamics simulations for aerodynamic calculations.
- **Driveline Efficiency ($\eta = 0.92$):**
Accounts for losses in the transmission, bearings, shafts, and other mechanical components between the power source (engine or motor) and the wheels. Toyota's e-CVT system exhibits high mechanical efficiency; therefore, a value of 92% is representative of its low-loss power transfer characteristics.

Together, these parameters allow the simulation to approximate the real-world longitudinal performance of the Toyota Prius, providing a valid and reliable foundation for analysing hybrid energy flows, fuel consumption, regenerative braking effectiveness, and control strategy behaviour.

4.3.2. Vehicle Model

The vehicle model defines how the vehicle responds to propulsion and braking forces along a 1-D longitudinal axis. In a HEV, the EMS relies on this model to determine how much mechanical power must be supplied (or absorbed) by the engine and/or electric machine at every timestep. Even if the model is simplified, it captures all major physical effects relevant to energy use, which is essential for comparing EMS strategies.

In a real vehicle, required traction power depends on:

- aerodynamic drag
- rolling resistance
- road grade
- inertial acceleration
- drivetrain losses

The EMS must know **how much power demand** the wheels impose on the powertrain before deciding how to split power between the engine, motor, and battery.

Thus, the vehicle model serves as the **load generator** for the EMS.

The simulation follows the standard longitudinal dynamic equation:

$$F_{\text{traction}} = F_{\text{aero}} + F_{\text{roll}} + F_{\text{grade}} + m a$$

Figure 5. Longitudinal Dynamic equation, created in Microsoft Word by Illia Novikov.

And required mechanical power:

$$P = F \cdot v$$

Figure 6. Power equation, created in Microsoft Word by Illia Novikov.

The model is simple enough for fast simulation but detailed enough to reflect realistic HEV behaviour.

1. Aerodynamic Drag

```
def aero_drag_force(self, v_mps):  
    return 0.5 * self.rho * self.Cd * self.A * v_mps * v_mps
```

Figure 7. Source: Screen capture of 'hev_project' './src/models/vehicle.py' from VS Code, created by Illia Novikov.

Aerodynamic drag grows quadratically with speed:

$$F_d = \frac{1}{2} \rho C_d A v^2$$

Figure 8. Aerodynamic Drag equation, created in Microsoft Word by Illia Novikov.

Where:

- ρ – air density (kg/m³)
- C_d – aerodynamic drag coefficient (dimensionless)
- A – vehicle frontal area (m²)
- v – vehicle velocity (m/s)

Drag increases with the square of velocity, meaning it becomes the dominant resistive force during high-speed driving. Including this term is essential, as it directly influences the shaft power demand calculated by the Vehicle model. This affects the EMS decision-making process: higher aerodynamic loads increase propulsion demand, engine usage, and battery discharge rates. The formula is therefore a critical component of the powertrain model used for energy management evaluation. This is a dominant force at high speeds (60-120 km/h), heavily influencing fuel use.

Why we model it:

- To create realistic propulsion power demand
- To ensure high-speed cruising load is realistic (important for SOC behaviour)

2. Rolling Resistance

```
def rolling_force(self):  
    # approximation: constant rolling at all speeds  
    return self.m * self.g * self.Cr
```

Figure 9. Source: Screen capture of 'hev_project' './src/models/vehicle.py' from VS Code, created by Illia Novikov.

Rolling resistance is approximately constant over speed:

$$F_r = mgC_r$$

Figure 10. Rolling Resistance equation, created in Microsoft Word by Illia Novikov.

Where:

- m – vehicle mass (kg)
- g – gravitational acceleration (m/s^2)
- C_r – rolling resistance coefficient (dimensionless)

This expression assumes rolling resistance is approximately constant over normal operating speeds, which is sufficiently accurate for longitudinal HEV simulations. The coefficient C_r aggregates tire properties, road texture, and tire inflation level. Rolling resistance contributes a nearly constant resistive force opposing motion. Although smaller than aerodynamic drag at higher speeds, it significantly affects low-speed driving and urban cycles, where hybrid vehicles operate frequently. Including this term allows the model to capture realistic power demand during city conditions, influencing both engine load and battery discharge behaviour within the EMS.

Why we model it:

- It dominates low-speed energy consumption (urban cycles)
- Even EVs spend most energy in cities fighting rolling resistance

3. Grade Force

```
def grade_force(self, slope_deg=0.0):  
    return self.m * self.g * math.sin(math.radians(slope_deg))
```

Figure 11. Source: Screen capture of 'hev_project' './src/models/vehicle.py' from VS Code, created by Illia Novikov.

Climbing or descending hills adds/subtracts load:

$$F_g = mg \sin(\theta)$$

Figure 12. Grade Force equation, created in Microsoft Word by Illia Novikov.

Where:

- m – mass of the object
- g – gravitational acceleration (m/s^2)
- θ – angle of the incline

When an object is on a hill, gravity pulls straight downward, but only a part of that force pulls the object *down the slope*.

That part is $mg \sin(\theta)$.

This expression represents the rolling resistance force, which arises from tire deformation, road surface texture, and tire pressure. The coefficient C_r combines these effects into a single value and is generally treated as constant over typical driving speeds. Rolling resistance opposes vehicle motion with an almost constant magnitude. While it is smaller than aerodynamic drag at high speeds, it remains important during low-speed and stop-and-go urban driving – conditions common for hybrid vehicles. Including F_r in the model helps accurately capture the power demand on the engine and battery during such cycles, improving the realism of the HEV energy management simulation.

Why we model it:

- Grade significantly affects SOC and optimal engine operation
- Even a small 3-4% grade changes powertrain behaviour dramatically

4. Inertia and Traction Force

```
def traction_force(self, v_mps, acc_mps2=0.0, slope_deg=0.0):  
    F = (self.aero_drag_force(v_mps)  
         + self.rolling_force()  
         + self.grade_force(slope_deg)  
         + self.m * acc_mps2)  
  
    return F
```

Figure 13. Source: Screen capture of 'hev_project' './src/models/vehicle.py' from VS Code, created by Illia Novikov.

Acceleration adds an inertial term:

$$F_{\text{inertia}} = ma$$

Figure 14. Inertia equation, created in Microsoft Word by Illia Novikov.

Where:

- m – vehicle mass (kg)
- a – longitudinal acceleration (m/s^2)

The inertial force represents the additional tractive effort required whenever the vehicle speeds up. Unlike rolling resistance or aerodynamic drag, this force does not oppose motion but instead reflects the physical effort needed to increase kinetic energy. During positive acceleration, the power demand rises because the drivetrain must supply energy to accelerate the mass of the vehicle. This term is crucial for modelling transient events such as overtaking, merging, or stop-and-go driving. It directly affects engine load and battery discharge, making it essential for accurate powertrain and EMS behaviour prediction.

Why we model it:

- electric motors respond instantly – ideal for acceleration
- engines have slower dynamics – EMS must anticipate transitions

5. Wheel Power Required

```
def power_required_at_wheels(self, v_mps, acc_mps2=0.0, slope_deg=0.0):  
    #negative (when braking).  
  
    F = self.traction_force(v_mps, acc_mps2, slope_deg)  
    return F * v_mps
```

Figure 15. Source: Screen capture of 'hev_project' './src/models/vehicle.py' from VS Code, created by Illia Novikov.

Basic mechanical power:

$$P_{\text{wheels}} = F_{\text{traction}} \cdot v$$

Figure 16. Mechanical Power equation, created in Microsoft Word by Illia Novikov.

Where:

- F_{traction} – total tractive force demand (N)
- v – vehicle speed (m/s)

This expression represents the instantaneous power needed to overcome all resistive forces and achieve the desired motion. Since power is the product of force and velocity, higher speeds or greater traction force directly increase the power demand. In HEV simulations, this value determines how much power must be delivered by the engine-motor combination before accounting for drivetrain losses, making it a fundamental input for the EMS and overall powertrain energy flow analysis.

Why we model it:

- This raw wheel power is the physical load the powertrain must handle

6. Shaft Power (Powertrain Input Power)

```
def power_required_at_shaft(self, v_mps, acc_mps2=0.0, slope_deg=0.0):  
    P_wheels = self.power_required_at_wheels(v_mps, acc_mps2, slope_deg)  
    P_shaft = P_wheels / max(1e-9, self.driveline_eff)  
    return max(0.0, P_shaft)
```

Figure 17. Source: Screen capture of 'hev_project' './src/models/vehicle.py' from VS Code, created by Illia Novikov.

Drivetrain losses must be accounted for:

$$P_{\text{shaft}} = \frac{P_{\text{wheels}}}{\eta_{\text{driveline}}}$$

Figure 18. Shaft Power equation, created in Microsoft Word by Illia Novikov.

Where:

- P_{wheels} – mechanical power required at the wheels (W)
- $\eta_{\text{driveline}}$ – driveline efficiency (dimensionless), accounting for gearbox, differential, and mechanical losses

Because the drivetrain is not perfectly efficient, the powertrain must supply more power than what reaches the wheels. Dividing by the efficiency compensates for losses in gears, bearings, lubricants, and mechanical coupling elements. This ensures the model realistically represents power flow between the motor-engine system and the wheels. Accurately estimating shaft power is critical for determining engine load, electric motor torque demand, and battery discharge patterns, forming the basis for evaluating EMS behaviour under different driving conditions.

Why we model it:

- This value feeds directly into EMS thresholds

7. Braking Power

```
def braking_power(self, v_mps, acc_mps2=0.0, slope_deg=0.0):  
  
    P_wheels = self.power_required_at_wheels(v_mps, acc_mps2, slope_deg)  
    # P_wheels negative => braking energy available (abs value)  
    P_brake_wheels = max(0.0, -P_wheels)  
    # convert wheels => shaft (approx)  
    return P_brake_wheels / max(1e-9, self.driveline_eff)
```

Figure 19. Source: Screen capture of 'hev_project' './src/models/vehicle.py' from VS Code, created by Illia Novikov.

When wheel power < 0, the vehicle is decelerating:

$$P_{\text{brake}} = \frac{\max(0, -P_{\text{wheels}})}{\eta_{\text{driveline}}}$$

Figure 20. Brake Power equation, created in Microsoft Word by Illia Novikov.

Where:

- P_{wheels} – net power demand at the wheels (W)
- $\eta_{\text{driveline}}$ – driveline efficiency (dimensionless)
- A negative P_{wheels} indicates deceleration

When the computed wheel power becomes negative, the vehicle is slowing down due to deceleration, road slope, or external resistances. In this condition, the wheels are no longer demanding power – they are *returning* mechanical energy toward the drivetrain. The braking power is defined as the magnitude of this negative power flow. The model uses the expression to ensure braking power is only positive during deceleration and zero during acceleration or steady-state driving. Braking power is essential for distinguishing mechanical braking from regenerative braking. Accurate calculation allows the EMS to determine how much of the deceleration energy can be recovered by the electric motor. This directly influences battery charging, motor torque limits, and overall energy consumption of the vehicle.

Why we model it:

- Regenerative power goes to the electric motor
- The EMS decides how much to recover
- Battery model applies charging limits

4.3.3. Motor Model

The electric motor model describes how mechanical shaft power is converted into electrical power at the battery, and how much propulsion or regenerative braking torque the motor can realistically provide. In a hybrid electric vehicle, the EMS depends on this model to determine the motor's contribution to total power demand, evaluate regenerative braking potential, and maintain battery SOC within limits. Even though the model is simplified, it preserves all key behaviours necessary for HEV control strategy evaluation.

In a real HEV, the electric machine performs two fundamental functions:

- Motoring (propulsion): delivering positive mechanical power to assist or replace the engine
- Generating (regeneration): converting braking or downhill energy into electrical energy stored in the battery

To model these behaviours accurately enough for EMS decision-making, the motor model must capture:

- maximum allowable mechanical power
- motoring efficiency
- regenerative efficiency
- limits on charging and discharging power
- the sign convention linking battery power and mechanical shaft power

The EMS uses this information to determine how much torque the motor can provide at each timestep, whether regenerative braking is possible, and how much electrical power will be drawn from or returned to the battery. This makes the motor model the central link between the mechanical load from the vehicle and the electrical load seen by the battery.

The core relationship in the model is the conversion between mechanical shaft power P_{mech} and battery electrical power P_{batt} .

During motoring (propulsion):

$$P_{\text{batt}} = \frac{P_{\text{mech}}}{\eta_{\text{motoring}}}$$

Figure 21. Electrical Power equation during propulsion, created in Microsoft Word by Illia Novikov.

During regeneration (braking):

$$P_{\text{batt}} = P_{\text{mech}} \cdot \eta_{\text{regen}}$$

Figure 22. Electrical Power equation during regeneration, created in Microsoft Word by Illia Novikov.

Positive P_{batt} indicates battery discharge, while negative P_{batt} corresponds to charging. The model also limits mechanical power within the range:

$$-P_{\text{max}} \leq P_{\text{mech}} \leq P_{\text{max}}$$

Figure 23. Range of Mechanical Power equation, created in Microsoft Word by Illia Novikov.

reflecting realistic torque and thermal constraints of the electric machine.

Although simplified, this motor representation contains the essential physics governing energy flow in a hybrid system. It is computationally efficient, easy to integrate into a timestep-based simulation, and provides realistic motor behaviour needed for testing EMS strategies, evaluating regenerative braking performance, and analysing fuel-electricity trade-offs in the HEV powertrain.

1. Initialization

```
class Motor:
    def __init__(self,
                  P_max_kW=53.0,
                  motoring_eff=0.92,
                  regen_eff=0.65):
        self.P_max_W = P_max_kW * 1000.0
        self.motoring_eff = motoring_eff
        self.regen_eff = regen_eff
```

Figure 24. Source: Screen capture of 'hev_project' './src/models/motor.py' from VS Code, created by Illia Novikov.

This section defines the electric motor's key parameters:

- $P_{\max}$ – maximum mechanical power the motor can deliver or absorb, converted from kW to watts
- η_{motoring} – efficiency when the motor provides propulsion
- η_{regen} – efficiency when the motor recovers braking energy

These parameters reflect real-world limits of the Prius electric machine. Motoring efficiency is generally high ($\approx 90\%$), while regenerative efficiency is lower ($\approx 60\text{--}70\%$) because energy conversion during braking inherently involves additional losses.

A hybrid controller must always respect the motor's maximum power and know the correct energy flow between mechanical and electrical domains. Without these limits, the EMS could command unrealistically high torque or overestimate regenerative energy recovery.

2. Mechanical Power Limitation

```
def clamp_mechanical(self, P_mech_w):  
    return max(-self.P_max_w, min(self.P_max_w, P_mech_w))
```

Figure 25. Source: Screen capture of 'hev_project' './src/models/motor.py' from VS Code, created by Illia Novikov.

This function ensures the motor's mechanical power stays within its physical limits:

$$-P_{\max} \leq P_{\text{mech}} \leq P_{\max}$$

Figure 23. Range of Mechanical Power equation, created in Microsoft Word by Illia Novikov.

Where:

- Positive P_{mech} means motor provides propulsion
- Negative P_{mech} means motor absorbs braking energy (regen)

If the EMS requests more power than the motor can handle, the model “clamps” it to the nearest allowed value.

Why we model it:

- Electric machines have torque and thermal limits
- Regenerative braking cannot exceed motor capacity
- The EMS must not command infeasible power levels

This guarantees physical realism in the power split between engine, battery, and motor.

3. Mechanical-to-Battery Power Conversion

```
def mech_to_batt(self, P_mech_W):  
  
    if P_mech_W >= 0:  
        # battery must supply more due to motor inefficiency  
        return P_mech_W / max(1e-9, self.motoring_eff)  
    else:  
        # regen: only part of mechanical is recovered  
        return P_mech_W * self.regen_eff
```

Figure 26. Source: Screen capture of 'hev_project' './src/models/motor.py' from VS Code, created by Illia Novikov.

This method converts mechanical shaft power to battery-side electrical power, applying the correct efficiency depending on operating mode.

The battery must supply more power than delivered at the shaft, because inefficiencies produce heat.

Meaning:

- If the motor outputs 20 kW with 92% efficiency
- The battery actually delivers ≈ 21.7 kW

Only a fraction of braking power becomes electrical energy due to:

- inverter losses
- copper losses
- battery internal resistance

Meaning:

- If -30 kW of braking energy is available
- Battery receives ≈ 19.5 kW

Why we model this:

- Determines how much braking energy can be stored
- Prevents unrealistic 100% efficient regeneration
- Ensures the battery SOC evolves correctly

4.3.4. Engine Model

The engine model represents how the ICE of the hybrid powertrain produces mechanical power and consumes fuel. For the EMS, this model determines the energetic cost of using the engine: how much mechanical power can be delivered at a given time and how much fuel that power requires. Although simplified, the model captures the key physical characteristics that govern engine behaviour and fuel consumption, making it suitable for energy-based comparison of EMS strategies.

In a real hybrid vehicle, engine operation is defined by several essential constraints:

- the engine can only produce positive mechanical power (it cannot absorb energy or brake the vehicle)
- its power output is limited by a rated maximum power (72 kW for the 2020 Toyota Prius engine)
- fuel consumption scales with mechanical power through the engine's Brake Specific Fuel Consumption (BSFC)
- the engine requires idle fuel even when producing nearly zero power,
- and the engine can be fully shut off (engine-off operation) to save fuel during light load or electric driving

The EMS must account for all these characteristics before deciding whether the engine should run and how much power it should contribute. Since fuel consumption depends strongly on engine load, knowing the precise mechanical power requirement and resulting fuel flow rate is crucial for optimizing hybrid operation.

The model computes the consumed fuel mass based on the standard BSFC definition:

$$\dot{m}_f = \frac{P_{kW} \cdot BSFC}{3600}$$

Figure 27. Consumed Fuel Mass equation, created in Microsoft Word by Illia Novikov.

Where BSFC is measured in grams per kWh. This mass flow is then converted to a volumetric rate (L/s) using the fuel density. Thus, the engine model provides two essential outputs to the EMS: the maximum available mechanical power at the crankshaft and the instantaneous fuel consumption associated with a given power level. Despite its simplicity, the model is detailed enough to reflect the key efficiency behaviour of the Toyota Prius engine, enabling accurate simulation of hybrid power-split strategies.

1. Initialization

```
class Engine:
    def __init__(self,
                  P_max_kw=72.0,
                  bsfc_g_per_kwh=240.0,
                  fuel_density_kg_per_l=0.745):
        # max mechanical power (W)
        self.P_max_W = P_max_kw * 1000.0
        # BSFC: grams of fuel per kWh mechanical (typical mid/peak values)
        self.bsfc = bsfc_g_per_kwh
        # fuel density (kg/L)
        self.fuel_density = fuel_density_kg_per_l

        # idle fuel expressed as small baseline when engine_on and producing near-zero mechanical
        self.idle_fuel_L_per_h = 0.9 # L/h typical idle
        self.idle_fuel_L_per_s = self.idle_fuel_L_per_h / 3600.0
```

Figure 28. Source: Screen capture of 'hev_project' './src/models/engine.py' from VS Code, created by Illia Novikov.

The initialization block defines the key physical parameters that govern engine behaviour. These parameters determine the engine's maximum mechanical capability and its fuel consumption characteristics.

Parameter meanings

- **P_max_kw**
Maximum mechanical power the engine can output (72 kW for a Toyota Prius engine). It is converted to watts
- **bsfc_g_per_kwh**
BSFC, given in grams per kWh. This expresses how much fuel mass is required to produce mechanical work
- **fuel_density_kg_per_l**
Used to convert fuel mass consumption into volume (L/s), which is the typical way to report fuel usage

This section defines all physical constants needed for fuel flow calculations. The EMS relies on this data to evaluate whether using the engine is energetically beneficial compared to running on electricity.

2. Power Limiting

```
def clamp_power(self, P_req_W):  
    return max(0.0, min(self.P_max_W, P_req_W))
```

Figure 29. Source: Screen capture of 'hev_project' './src/models/engine.py' from VS Code, created by Illia Novikov.

The engine can only output power within its physical limits:

$$0 \leq P_{\text{engine}} \leq P_{\text{max}}$$

Figure 30. Power Limits equation, created in Microsoft Word by Illia Novikov.

The lower limit is 0, because the engine cannot absorb power (it cannot regenerate). The upper limit is maximum rated power (self.P_max_W). Without clamping, EMS or vehicle dynamics could request unrealistic power levels, causing simulation instability.

This function ensures:

- No negative power (ICE cannot brake the vehicle)
- No exceeding the rated mechanical capability

3. Fuel Flow Calculation

```
def fuel_flow_L_per_s(self, P_mech_W, engine_on=True):  
    if not engine_on:  
        return 0.0  
  
    P_mech_W = max(0.0, P_mech_W)  
    if P_mech_W < 1.0:  
        return self.idle_fuel_L_per_s  
  
    # mechanical power in kW  
    P_kW = P_mech_W / 1000.0  
  
    # fuel mass flow (g/s) = P_kW * BSFC (g/kWh) / 3600  
    g_per_s = (P_kW * self.bsfc) / 3600.0  
  
    # convert g/s to L/s  
    kg_per_s = g_per_s / 1000.0  
    L_per_s = kg_per_s / max(1e-9, self.fuel_density)  
    return L_per_s
```

Figure 31. Source: Screen capture of 'hev_project' './src/models/engine.py' from VS Code, created by Illia Novikov.

If EMS turns the engine off, fuel flow is exactly zero, regardless of requested power. Even when mechanical power is near zero, the gasoline engine still consumes fuel to keep rotating. This ensures realistic idle behaviour.

Meaning:

This transforms mechanical power demand into fuel consumption based on:

- engine thermodynamic efficiency
- energy density of fuel
- BSFC characteristics

Why we model it:

- computes realistic fuel usage based on BSFC
- includes idle fuel consumption
- converts power demand into fuel volume with proper unit consistency
- restricts power to physical limits,
- allows engine-off operation with zero fuel flow

4.3.5. Battery Model

The battery model represents the energy storage system of the hybrid vehicle and determines how electrical power flows into and out of the high-voltage traction battery. In a hybrid powertrain, the battery directly influences the decisions of the EMS because battery state and limitations determine whether the electric motor can provide propulsion, regenerative braking, or must rely on the engine as the primary energy source.

A battery in an HEV operates within a controlled SOC range to ensure long life, optimal efficiency, and safety. Most HEVs (such as the Toyota Prius) use relatively small batteries (0.75–1.3 kWh usable capacity), cycled between 40-80% SOC. The model replicates this by defining SOC limits, voltage-related inefficiencies, and maximum charge/discharge power.

The battery model serves several key functions:

- **Energy tracking:**
It maintains the current stored energy E and converts between electrical power, energy, and SOC.
- **Power limitation:**
Batteries have restricted charge and discharge limits. This prevents unrealistic power flows and forces the EMS to rely on the engine when battery power is insufficient.
- **Efficiency modelling:**
Charging and discharging have different efficiencies due to resistive losses and electrochemical processes.
- **SOC constraints:**
If SOC reaches upper or lower bounds, the model returns surplus or deficit power, instructing the EMS that additional handling is required (engine must cover deficit or regenerative energy must be curtailed).

Even though simplified, the model captures all essential physical behaviours required for EMS strategy development: limited power availability, asymmetric efficiency, SOC change over time, and realistic energy counting.

1. Initialization

```
class Battery:
    def __init__(self,
                  capacity_kwh=0.75,
                  soc_init=0.6,
                  soc_min=0.4,
                  soc_max=0.8,
                  eta_ch=0.95,
                  eta_dis=0.95,
                  max_discharge_kw=25.0,
                  max_charge_kw=20.0):
        self.capacity_Wh = capacity_kwh * 1000.0
        self.E_Wh = self.capacity_Wh * soc_init
        self.soc_min = soc_min
        self.soc_max = soc_max
        self.eta_ch = eta_ch
        self.eta_dis = eta_dis
        self.max_discharge_W = max_discharge_kw * 1000.0
        self.max_charge_W = max_charge_kw * 1000.0
```

Figure 32. Source: Screen capture of 'hev_project' './src/models/battery.py' from VS Code, created by Illia Novikov.

Parameter meanings:

- capacity_kwh – usable battery capacity
Converted to watt-hour

$$\text{capacity}_{Wh} = \text{capacity}_{kWh} \times 1000$$

Figure 33. Battery Capacity equation, created in Microsoft Word by Illia Novikov.

- soc_init – initial SOC
Used to set initial energy:

$$E_{Wh} = SOC_{init} \cdot \text{capacity}_{Wh}$$

Figure 34. Initial SOC equation, created in Microsoft Word by Illia Novikov.

- soc_min / soc_max – operating SOC window
Typical hybrid strategy: 40-80% SOC range
- eta_ch / eta_dis – charging and discharging efficiencies

Why we model this:

This block sets all the fundamental battery characteristics that the EMS must respect:

- how much energy is available
- how much power can flow in or out
- when the engine must take over
- when regenerative braking must be limited or disabled

2. SOC Property

```
@property
def soc(self):
    return self.E_Wh / max(1e-9, self.capacity_Wh)
```

Figure 35. Source: Screen capture of 'hev_project' './src/models/battery.py' from VS Code, created by Illia Novikov.

Dynamically computes SOC from current stored energy:

$$SOC = \frac{E_{Wh}}{capacity_{Wh}}$$

Figure 36. SOC equation, created in Microsoft Word by Illia Novikov.

Where:

- E_{Wh} – current energy in the battery (in Wh)
- $Capacity_{Wh}$ – maximum possible energy the battery can store (in Wh)

The $\max(1e-9, \dots)$ term prevents division by zero.

Why we model this:

- Battery Limits Power Output
- Control Logic Depends on SOC

3. Battery Power Application

```
def apply_battery_power(self, P_batt_w, dt_s):  
    # enforce instantaneous power limits  
    P_applied = P_batt_w
```

Figure 37. Source: Screen capture of 'hev_project' './src/models/battery.py' from VS Code, created by Illia Novikov.

The main function that updates battery energy and returns surplus/deficit power.

Meaning:

The EMS requests battery power P_{batt} .

This function verifies whether that request is physically achievable and updates the stored energy accordingly.

It returns:

- 0 when everything is valid,
- positive surplus power (W) when charging exceeds limits,
- negative deficit power (W) when discharging exceeds limits.

The EMS then uses these signals to adjust engine power.

4. Power Limit Enforcement

```
if P_batt_W > self.max_discharge_W:
    # request to discharge more than max -> deficit (W) to be covered by engine
    deficit = P_batt_W - self.max_discharge_W
    P_applied = self.max_discharge_W
elif P_batt_W < -self.max_charge_W:
    # request to charge beyond max -> surplus (wasted)
    surplus = P_batt_W + self.max_charge_W # negative + positive => negative surplus value
    P_applied = -self.max_charge_W
else:
    surplus = 0.0
    deficit = 0.0
```

Figure 38. Source: Screen capture of 'hev_project' './src/models/battery.py' from VS Code, created by Illia Novikov.

What this means:

- If EMS wants more discharge than allowed:
The battery gives only its maximum, engine must cover the rest.
- If EMS wants more charging than allowed:
Excess regenerative power becomes surplus (lost).

Why we model this:

This reflects real batteries, which have:

- thermal limits
- voltage constraints
- current limitations

Hybrid systems heavily rely on these limitations – this is essential input for EMS decisions.

5. Energy Update (Charging/Discharging)

```
# energy change in Wh. For discharging (P_applied>0) battery loses more due to inefficiency
if P_applied >= 0:
    delta_Wh = (P_applied * dt_s) / 3600.0 / max(1e-9, self.eta_dis)
else:
    # charging: stored energy is less due to charge efficiency
    delta_Wh = (P_applied * dt_s) / 3600.0 * max(0.0, self.eta_ch)

self.E_Wh -= delta_Wh
```

Figure 38. Source: Screen capture of 'hev_project' './src/models/battery.py' from VS Code, created by Illia Novikov.

Energy change is computed as:

$$\Delta E = \frac{P \cdot \Delta t}{3600}$$

Figure 39. Energy Change equation, created in Microsoft Word by Illia Novikov.

- Discharging (positive power):
Battery loses extra energy due to inefficiency.
- Charging (negative power):
Battery stores only a fraction.

This models the realistic asymmetric behaviour of Lithium batteries.

6. SOC Upper Bound Handling

```
# upper bound
if self.E_wh > self.capacity_wh * self.soc_max:
    surplus_wh = self.E_wh - self.capacity_wh * self.soc_max
    self.E_wh = self.capacity_wh * self.soc_max
    # convert Wh surplus -> W over dt_s
    return surplus_wh * 3600.0 / max(1e-9, dt_s)
```

Figure 40. Source: Screen capture of 'hev_project' './src/models/battery.py' from VS Code, created by Illia Novikov.

Meaning:

If charging pushes SOC above allowed maximum:

- extra energy is removed
- returned as surplus power (W)

This is exactly how hybrids terminate regenerative braking when SOC becomes high.

7. SOC Lower Bound Handling

```
# lower bound
if self.E_wh < self.capacity_wh * self.soc_min:
    deficit_wh = self.capacity_wh * self.soc_min - self.E_wh
    self.E_wh = self.capacity_wh * self.soc_min
    return -deficit_wh * 3600.0 / max(1e-9, dt_s)

return 0.0
```

Figure 41. Source: Screen capture of 'hev_project' './src/models/battery.py' from VS Code, created by Illia Novikov.

Meaning:

If discharging pushes SOC below allowed minimum:

- battery is clamped to minimum
- a deficit power (negative) is returned, signalling EMS must power the vehicle with the engine

4.3.6. Models Test

```
1 # tests/run_model_test.py
2 from src.models.engine import Engine
3 from src.models.motor import Motor
4 from src.models.battery import Battery
5 from src.models.vehicle import Vehicle
6
7 def run_quick_checks():
8     eng = Engine()
9     mot = Motor()
10    bat = Battery()
11    veh = Vehicle()
12
13    # Vehicle: power at 20 m/s (~72 km/h)
14    v = 20.0
15    acc = 0.5
16    P_shaft = veh.power_required_at_shaft(v, acc)
17    print(["Power required at shaft (kW):", P_shaft/1000.0])
18
19    # Engine: fuel for 20 kW mechanical
20    P_eng = 20000.0
21    print("Engine fuel (L/s) for 20kW:", eng.fuel_flow_L_per_s(P_eng))
22
23    # Motor: electrical draw for 30 kW mech
24    P_mot = 30000.0
25    print("Motor electrical (W) for 30kW mech:", mot.mech_to_batt(P_mot))
26
27    # Battery: apply 10 kW discharge for 10 s
28    print("Initial SOC:", bat.soc)
29    res = bat.apply_battery_power(10000.0, 10.0)
30    print("Battery apply 10kW x10s returned:", res, "SOC now:", bat.soc)
31    # try regen beyond charge limit
32    res2 = bat.apply_battery_power(-50000.0, 1.0) # -50kW charging request (bigger than max charge)
33    print("Attempted huge regen returned:", res2, "SOC:", bat.soc)
34
35 if __name__ == '__main__':
36     run_quick_checks()
```

Figure 42. Source: Screen capture of 'hev_project' './src/tests/run_model_test.py' from VS Code, created by Illia Novikov.

The test script **run_model_test.py** provides a quick functional check of all four powertrain models to confirm that they behave realistically before being used in the full hybrid simulation. The script initializes the Engine, Motor, Battery, and Vehicle models with default Prius-based parameters, and then applies several controlled inputs to verify that each subsystem responds consistently with physical expectations.

The first test computes the mechanical power required at the wheels when the vehicle travels at 20 m/s with a small acceleration.

Next, the engine model is asked to deliver 20 kW.

The motor model is evaluated by requesting 30 kW of mechanical shaft power.

Battery behaviour is tested in two steps. First, discharging 10 kW for 10 seconds. Then an oversized regenerative braking request of –50 kW is applied.

Output:

```
(.venv) PS C:\Users\Illia\Desktop\DP\project\hev_project> python -m src.tests.run_model_test
Power required at shaft (kW): 21.173846739130433
Engine fuel (L/s) for 20kW: 0.0017897091722595079
Motor electrical (W) for 30kW mech: 32608.695652173912
Initial SOC: 0.6
Battery apply 10kW x10s returned: 0.0 SOC now: 0.5610136452241715
Attempted huge regen returned: 0.0 SOC: 0.5680506822612086
```

Figure 43. Source: Screen capture of 'hev_project' './src/tests/run_model_test.py' console from VS Code, created by Illia Novikov.

The printed result, ≈ 21.17 kW required at shaft, confirms that the vehicle model correctly combines drag, rolling resistance, inertia, and driveline losses into a realistic power demand.

Fuel-flow equation calculates a consumption rate of 0.00179 L/s, which is typical for a small hybrid engine at moderate load. This confirms that the BSFC-based fuel model is functioning correctly.

Due to efficiency losses, the corresponding electrical demand is higher than ≈ 32.6 kW, showing that the efficiency mapping (motoring vs. regen) is applied properly.

The need reduces SOC from 0.60 to 0.561, confirming correct energy accounting and discharge efficiency.

The model tries the charge power to its limit, slightly increasing SOC to 0.568, demonstrating that charge limits and SOC boundaries are enforced.

Together, these results show that all subsystem models (vehicle, engine, motor, battery) are stable, realistic, and ready for integration into the EMS simulation.

4.3.7. Controller

```
# src/controllers/rule_based.py
import time

class RuleBasedController:

    def __init__(self,
                 soc_min=0.4,
                 soc_max=0.8,
                 motor_max_W=25000.0,
                 engine_max_W=72000.0,
                 engine_min_on_time_s=8.0):
        self.soc_min = soc_min
        self.soc_max = soc_max
        self.motor_max_W = motor_max_W
        self.engine_max_W = engine_max_W
        self.engine_min_on_time_s = engine_min_on_time_s

        self.engine_on = False
        self._engine_last_switch = -1e9

    def _turn_engine_on(self, t_now):
        if not self.engine_on:
            self.engine_on = True
            self._engine_last_switch = t_now

    def _can_turn_engine_off(self, t_now):
        # enforce minimum on-time to prevent hunting
        if not self.engine_on:
            return True
        return (t_now - self._engine_last_switch) >= self.engine_min_on_time_s

    def _turn_engine_off(self, t_now):
        if self.engine_on and self._can_turn_engine_off(t_now):
            self.engine_on = False
            self._engine_last_switch = t_now
```

Figure 44. Source: Screen capture of 'hev_project' './src/controllers/rule_based.py' from VS Code, created by Illia Novikov.

The small internal methods of the controller serve one main purpose: to manage the engine's ON/OFF state safely and realistically. They keep track of when the engine last switched, make sure it can only turn off after a minimum amount of time, and prevent rapid shifting between ON and OFF when load fluctuates. These helper functions do not decide power splits themselves – instead, they ensure the engine behaves like a real hybrid system component, where switching has costs, delays, and constraints. Without these functions, the controller would constantly flick the engine on and off, which would be unrealistic and mechanically damaging in a real vehicle.

Main function():

```
def decide(self, P_req_shaft_W, soc, t_now):
    # if braking/regenerative demand (handled externally), controller is consulted with P_req_shaft_W==0
    P_req = max(0.0, float(P_req_shaft_W))

    # basic SOC rules
    if soc <= self.soc_min:
        # force engine on to protect battery
        self._turn_engine_on(t_now)
    elif soc >= self.soc_max:
        # allow engine off if min on time satisfied
        if self._can_turn_engine_off(t_now):
            self._turn_engine_off(t_now)

    # Decision logic:
    # - Low demand: motor only (if SOC allows)
    # - Medium demand: if SOC healthy -> motor only; else engine on
    # - High demand: engine on + motor assist up to motor_max
    LOW_W = 8000.0 # below ~8 kW prefer motor-only
    MID_W = 25000.0 # above ~25 kW engine assist recommended

    P_engine = 0.0
    P_motor = 0.0
```

Figure 45. Source: Screen capture of 'hev_project' './src/controllers/rule_based.py' from VS Code, created by Illia Novikov.

The `decide()` method is the main function of the controller and determines how much power should come from the engine and how much from the electric motor, and whether the engine should be running. It combines battery SOC rules, driver power demand, and hardware power limits into one coherent control strategy. First, the method uses SOC thresholds to protect the battery: low SOC forces the engine to run, while high SOC allows the engine to shut down when appropriate. Then it analyses the requested shaft power and places it into three broad demand categories: low, medium, and high. Low demand prefers electric-only operation, medium demand uses the motor when SOC is healthy but engages the engine if SOC is weak or the motor saturates, and high demand always requires engine power with motor assist. Throughout this process, the engine and motor outputs are taken by their maximum capabilities, ensuring realistic behaviour. The function ultimately returns a structured decision containing mechanical engine power, motor power, and the engine's ON/OFF state, which other parts of the simulation use to compute fuel use, battery changes, and vehicle performance.

Main logic:

```
if P_req <= LOW_W:
    # Small loads handled by motor only if SOC > min
    if soc > self.soc_min + 0.02:
        P_motor = min(P_req, self.motor_max_W)
        P_engine = 0.0
    else:
        # battery low -> engine run to supply low-power loads and charge
        self._turn_engine_on(t_now)
        P_engine = min(P_req, self.engine_max_W)
        P_motor = max(0.0, P_req - P_engine)

elif P_req <= MID_W:
    # medium demand
    if soc > (self.soc_min + 0.05):
        # prefer motor-only when possible (depending on motor_max)
        if P_req <= self.motor_max_W:
            P_motor = P_req
            P_engine = 0.0
        else:
            # motor saturates -> engine supplies remainder
            P_motor = self.motor_max_W
            P_engine = P_req - P_motor
            self._turn_engine_on(t_now)
    else:
        # SOC low -> engine supplies most
        self._turn_engine_on(t_now)
        # choose an engine operating power to keep engine in reasonable range (attempt to run moderate load)
        desired_engine_frac = 0.6
        P_engine = min(self.engine_max_W, desired_engine_frac * P_req)
        P_motor = max(0.0, P_req - P_engine)
        P_motor = min(P_motor, self.motor_max_W)

else:
    # high demand: use engine (up to max) + motor assist
    self._turn_engine_on(t_now)
    P_engine = min(P_req, self.engine_max_W)
    P_motor = max(0.0, P_req - P_engine)
    P_motor = min(P_motor, self.motor_max_W)
```

Figure 46. Source: Screen capture of 'hev_project' './src/controllers/rule_based.py' from VS Code, created by Illia Novikov.

Return:

```
return {
    "P_engine_mech": float(max(0.0, P_engine)),
    "P_motor_mech": float(P_motor),
    "engine_on": bool(self.engine_on)
}
```

Figure 47. Source: Screen capture of 'hev_project' './src/controllers/rule_based.py' from VS Code, created by Illia Novikov.

Method returns a **dictionary** that contains the controller's decision about how to split power between the engine and the motor, and whether the engine should be ON or OFF.

In other words, calling `decide(P_req_shaft_W, soc, t_now)` gives a complete power allocation decision for the hybrid system at that moment.

4.4. Simulation

Main function:

```
def run_simulation(drive_cycle_path=None, save_prefix="run_fixed"):  
    # instantiate models  
    vehicle = Vehicle()  
    engine = Engine() # fuel_flow_L_per_s(P_mech_W, engine_on)  
    motor = Motor()  
    battery = Battery()  
  
    controller = RuleBasedController(soc_min=battery.soc_min,  
                                     soc_max=battery.soc_max,  
                                     motor_max_W=motor.P_max_W,  
                                     engine_min_on_time_s=5.0)  
  
    t_arr, v_mps = load_drive_cycle(drive_cycle_path, dt=SIM_PARAMS["dt"])  
    N = len(t_arr)  
    dt = SIM_PARAMS["dt"]  
  
    rows = []  
    cumulative_fuel_L = 0.0  
  
    for i in range(N):  
        t_now = float(t_arr[i])  
        v = float(v_mps[i])  
        # acceleration forward difference  
        if i == 0:  
            acc = 0.0  
            v_prev = v  
        else:  
            v_prev = float(v_mps[i-1])  
            acc = (v - v_prev) / dt  
  
        # positive propulsion demand at shaft (W)  
        P_req_shaft = vehicle.power_required_at_shaft(v, acc, slope_deg=0.0) # >= 0  
  
        # available braking mechanical power at wheels (positive value in W)  
        P_brake_mech_wheels = vehicle.braking_power(v, acc, slope_deg=0.0) # >= 0 (wheels)  
        # convert wheel braking power to shaft braking (approx divide by driveline_eff)  
        P_brake_available_shaft = P_brake_mech_wheels / max(1e-9, vehicle.driveline_eff)
```

Figure 48. Source: Screen capture of 'hev_project' './src/simulation/simulate.py' from VS Code, created by Illia Novikov.

The simulation begins by preparing all essential HEV components: the vehicle dynamics model, the internal combustion engine, the electric motor, the battery, and the rule-based controller responsible for deciding the power split. Before the actual simulation loop starts, the program loads a drive cycle, which defines how the vehicle speed changes over time. If no external file is provided, the simulator uses a synthetic cycle that imitates typical urban and short highway driving. This gives you an immediate way to test the control logic. Later, when we supply a WLTP CSV, the simulator will instead use the official cycle data to evaluate performance under realistic certification conditions.

After the models and drive cycle are prepared, the main simulation loop runs forward in time, one second at a time. At each timestep, the simulator reads the current speed, estimates acceleration, and computes the mechanical power required at the vehicle shaft using the physics-based vehicle model. If the vehicle is decelerating, the simulator also calculates how much mechanical braking power is available so the motor can potentially recover energy through regenerative braking.

Main logic:

```
# Controller decision
decision = controller.decide(P_req_shaft, battery.soc, t_now)

P_engine_mech = max(0.0, float(decision.get("P_engine_mech", 0.0)))
P_motor_mech = float(decision.get("P_motor_mech", 0.0))
engine_on = bool(decision.get("engine_on", False))

P_motor_mech = max(-motor.P_max_W, min(motor.P_max_W, P_motor_mech))

# If there is braking energy available, we can't regenerate more than available braking power
if P_motor_mech < 0:
    regen_mech_abs = min(abs(P_motor_mech), P_brake_available_shaft)
    P_motor_mech = -regen_mech_abs

# Now convert motor mechanical to battery electrical power
P_batt_from_motor_W = motor.mech_to_batt(P_motor_mech)

# Enforce battery instantaneous power limits inside battery.apply_battery_power
surplus_or_deficit_W = battery.apply_battery_power(P_batt_from_motor_W, dt)

# Handle battery deficits/surplus:
# If battery couldn't discharge enough (deficit < 0), engine must supply the missing mechanical power.
if surplus_or_deficit_W < 0:
    # deficit (negative): battery couldn't supply requested discharge (W)
    needed_from_engine_W = -surplus_or_deficit_W
    # engine must increase mechanical contribution by this amount at shaft
    P_engine_mech += needed_from_engine_W

# Compute fuel for engine mechanical power
fuel_L_per_s = engine.fuel_flow_L_per_s(P_engine_mech, engine_on=engine_on)
fuel_this_step_L = fuel_L_per_s * dt
cumulative_fuel_L += fuel_this_step_L
```

Figure 49. Source: Screen capture of 'hev_project' './src/simulation/simulate.py' from VS Code, created by Illia Novikov.

The controller is then given the current power demand, battery state of charge, and current time. It returns a decision describing how much power the engine should deliver, how much the electric motor should provide, and whether the engine should remain ON or OFF. This decision reflects the hybrid energy management strategy implemented earlier.

Once the desired engine and motor power outputs are determined, the simulator enforces physical limits. Regenerative braking is capped by the actual braking energy available from the vehicle, and motor power is limited by its maximum rating. The motor's mechanical power is converted into battery electrical power, and the battery model updates its SOC while enforcing its own charge and discharge limits. If the battery cannot provide the required electrical power, the deficit is automatically shifted to the engine so that vehicle demand is always met.

Logs & Plots:

```
# Log row
rows.append({
    "t": t_now,
    "v_mps": v,
    "v_kph": v * 3.6,
    "acc_mps2": acc,
    "P_req_shaft_W": P_req_shaft,
    "P_brake_avail_shaft_W": P_brake_available_shaft,
    "P_engine_mech_W": P_engine_mech,
    "P_motor_mech_W": P_motor_mech,
    "P_batt_W": P_batt_from_motor_W,
    "SOC": battery.soc,
    "engine_on": engine_on,
    "fuel_total_L": cumulative_fuel_L
})

df = pd.DataFrame(rows)
out_csv = os.path.join(LOG_DIR, f"{save_prefix}_results.csv")
df.to_csv(out_csv, index=False)
print("Saved results to:", out_csv)

# Plotting
fig, axs = plt.subplots(5, 1, figsize=(10, 14), sharex=True)
axs[0].plot(df["t"], df["v_kph"]); axs[0].set_ylabel("Speed (km/h)"); axs[0].grid(True)
axs[1].plot(df["t"], df["P_req_shaft_W"] / 1000.0); axs[1].set_ylabel("P_req (kW)"); axs[1].grid(True)
axs[2].plot(df["t"], df["P_engine_mech_W"] / 1000.0, label="Engine"); axs[2].plot(df["t"], df["P_motor_mech_W"] / 1000.0, label="Motor");
axs[3].plot(df["t"], df["P_batt_W"] / 1000.0); axs[3].set_ylabel("Battery P (kW)"); axs[3].grid(True)
axs[4].plot(df["t"], df["SOC"]); axs[4].set_ylabel("SOC"); axs[4].set_xlabel("Time (s)"); axs[4].grid(True)
plt.tight_layout()
plot_file = os.path.join(PLOT_DIR, f"{save_prefix}_summary.png")
fig.suptitle("HEV Simulation (fixed regen)", y=1.02)
fig.savefig(plot_file, bbox_inches="tight")
print("Saved plot to:", plot_file)
plt.show()

print("Final fuel consumed (L):", df["fuel_total_L"].iloc[-1])
return df
```

Figure 50. Source: Screen capture of 'hev_project' './src/simulation/simulate.py' from VS Code, created by Illia Novikov.

After resolving power flows, the simulator calculates the fuel consumption for that timestep based on the engine's mechanical output and operating state. All relevant values – speed, acceleration, power flows, SOC, engine status, and cumulative fuel – are stored in a log. Once the full drive cycle is processed, the results are saved to a CSV file, and several summary plots are generated to visualize the behaviour of the hybrid system, including speed, power demand, engine/motor power split, battery power, and SOC evolution.

4.4.1. Synthetic Cycle

```
SIM_PARAMS = {
    "dt": 1.0, # seconds
    "T": 600.0, # simulation duration if no CSV provided
}

def load_drive_cycle(filepath=None, dt=1.0):
    #Return (t_array, v_mps_array). If no file, return a smooth synthetic cycle
    if filepath and os.path.isfile(filepath):
        df = pd.read_csv(filepath)
        if "speed_kph" in df.columns:
            v = df["speed_kph"].values / 3.6
        elif "speed_mps" in df.columns:
            v = df["speed_mps"].values
        elif "speed_kmh" in df.columns:
            v = df["speed_kmh"].values / 3.6
        else:
            raise ValueError("Drive cycle CSV must contain speed_kph or speed_mps column")
        t = df["time_s"].values if "time_s" in df.columns else np.arange(len(v)) * dt
        return t, v

    # synthetic smooth cycle (city + short highway)
    T = int(SIM_PARAMS["T"])
    t = np.arange(0, T, dt)
    v_kph = np.zeros_like(t, dtype=float)
    for i, ti in enumerate(t):
        if ti < 60:
            v_kph[i] = ti * (30/60) # accelerate 0->30
        elif ti < 120:
            v_kph[i] = 30
        elif ti < 150:
            v_kph[i] = 30 - (ti-120) # slow to stop
        elif ti < 200:
            v_kph[i] = 0
        elif ti < 260:
            v_kph[i] = (ti-200)*(50/60) # accelerate to 50
        else:
            v_kph[i] = 50
    return t, v_kph / 3.6
```

Figure 51. Source: Screen capture of 'hev_project' './src/simulation/simulate.py' from VS Code, created by Illia Novikov.

Before running the full hybrid simulation, the script first loads a drive cycle using the `load_drive_cycle()` function. If no external CSV file is provided, the simulator automatically generates a synthetic drive cycle based on the `SIM_PARAMS` settings, which define a 600-second test with a 1-second timestep. This synthetic cycle represents a smooth mix of city and short-highway driving: the vehicle accelerates up to 30 km/h, cruises, slows to a stop, remains stationary, accelerates again up to 50 km/h, and finally cruises at that speed. This built-in cycle allows us to immediately test the controller and vehicle models without requiring any real-world drive data. Once this synthetic cycle is simulated and the system behaviour is verified, we can then load an official WLTP drive cycle from a CSV file to observe how the hybrid controller performs under.

Synthetic Cycle Run & Results:

The synthetic cycle used in this simulation is a simple, smooth speed profile lasting 600 seconds, designed to mimic light urban driving followed by a brief highway section and long idle periods.

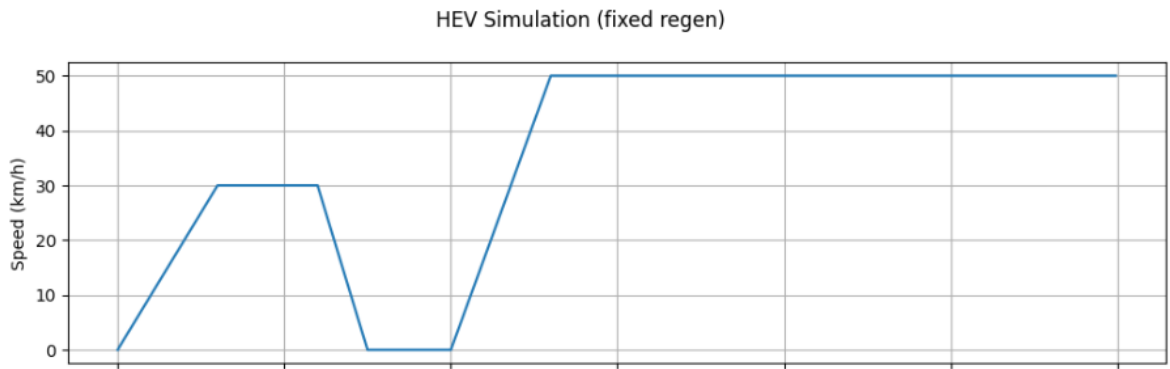


Figure 52. Source: Output plot generated by the script '`hev_project/src/simulation/simulate.py`' and exported automatically to '`./results/plots/run_fixed_summary.png`'. Created by Illia Novikov.

The vehicle first accelerates gradually up to about 30 km/h, cruises for a short while, slows back to a stop, and remains stationary. It then accelerates again to 50 km/h and cruises before finally returning to rest for the remainder of the simulation. Because the second half of the cycle contains very long idle intervals, the power demand becomes zero for extended durations, which strongly influences how the hybrid control strategy behaves.

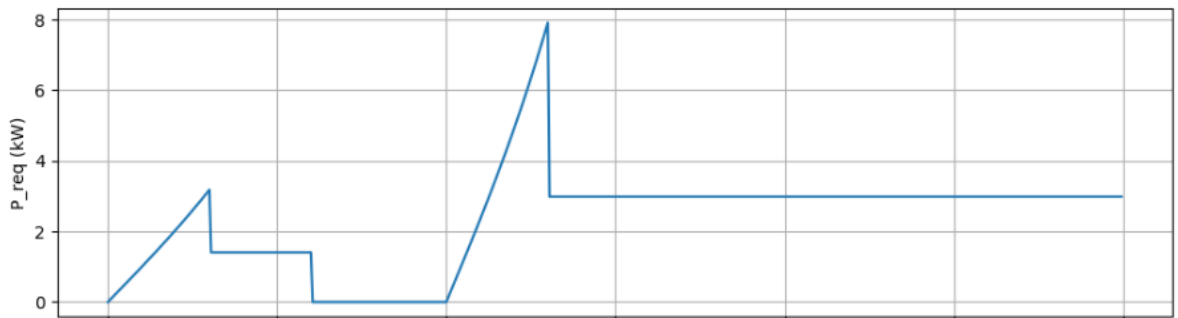


Figure 53. Source: Output plot generated by the script '`hev_project/src/simulation/simulate.py`' and exported automatically to '`./results/plots/run_fixed_summary.png`'. Created by Illia Novikov.

Throughout the drive, the required shaft power follows the changes in speed and acceleration. Power demand rises during the acceleration phases and reaches roughly 8 kW at the highest acceleration point, while during cruising at 30-50 km/h it stabilizes at only 2-3 kW. Whenever the vehicle is stationary, the required shaft power drops to zero. This transition between short periods of moderate power demand and long periods of zero power creates a situation where the electric motor is naturally chosen by the control algorithm.

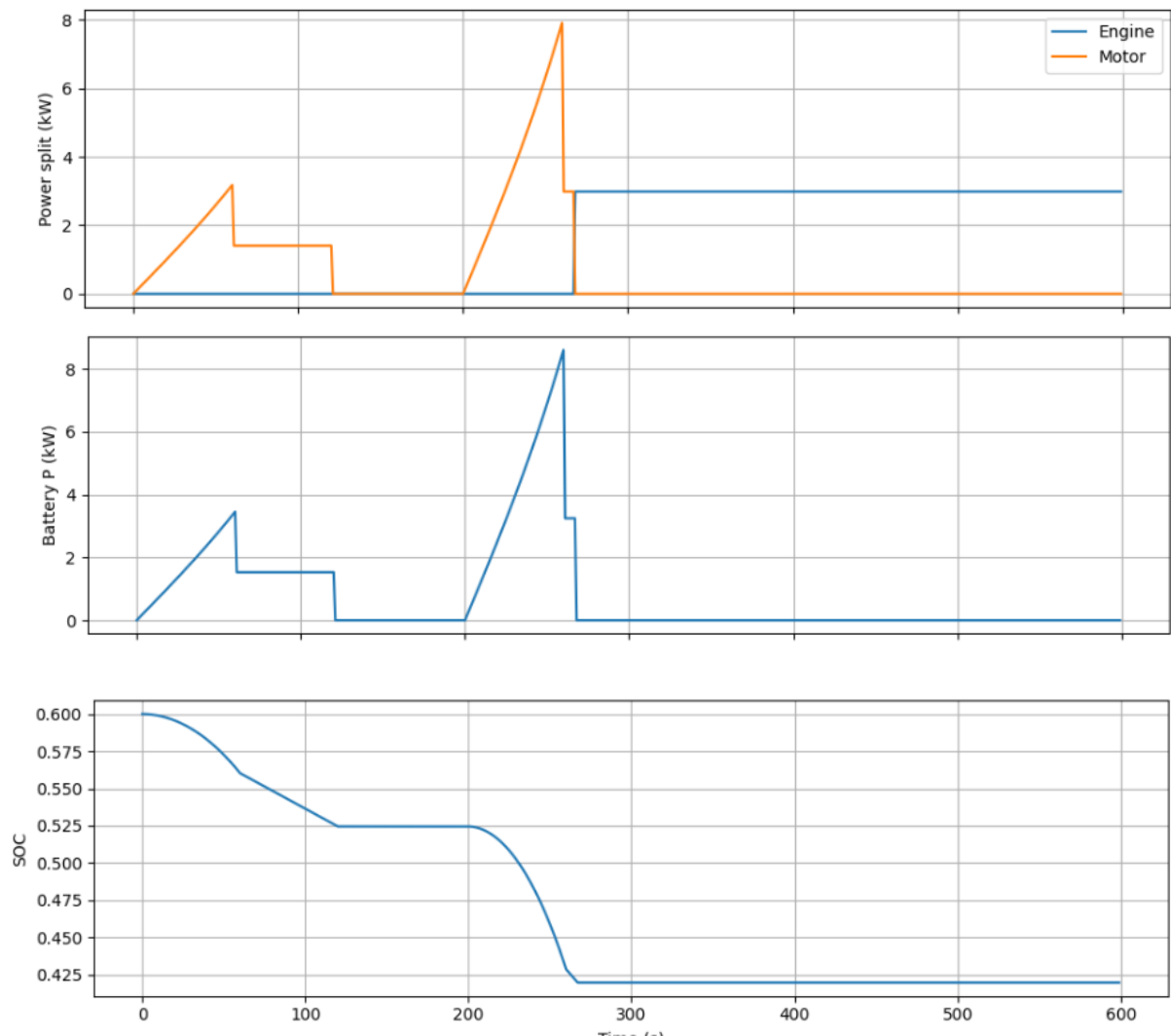


Figure 54. Source: Output plot generated by the script `'hev_project/src/simulation/simulate.py'` and exported automatically to `'./results/plots/run_fixed_summary.png'`. Created by Illia Novikov.

Because the battery begins at a healthy state of charge (around 60%), the rule-based controller initially chooses motor-only operation for the entire low-speed section. The electric motor supplies all traction during the early accelerations, which results in a gradual decline in SOC. When the vehicle reaches the stronger acceleration phase around 200-250 seconds, the combination of low SOC and increased power demand triggers the engine to turn on. Once engaged, the engine provides a steady mechanical power level of about 3 kW for an extended period. This is due to the controller's preference for operating the engine in a moderate, efficient region instead of switching it on and off rapidly. During the high-power acceleration peak, both the motor and engine contribute, with the motor providing the additional power beyond the engine's steady output.

As the simulation progresses, the SOC continues to fall until it reaches the minimum allowable threshold defined by the controller. Once this threshold is reached, the controller restricts battery discharge entirely: the motor no longer assists in propulsion, and the engine must remain on to ensure the battery does not drop further. After the acceleration phases end, the vehicle spends several hundred seconds at zero speed, so there is no further torque demand. During this prolonged idle time, both the motor and battery remain inactive, and the engine either remains on only to satisfy minimum-on-time conditions or turns off when allowed. Consequently, battery power remains nearly zero during the entire second half of the simulation, and SOC stabilizes at its lower limit.

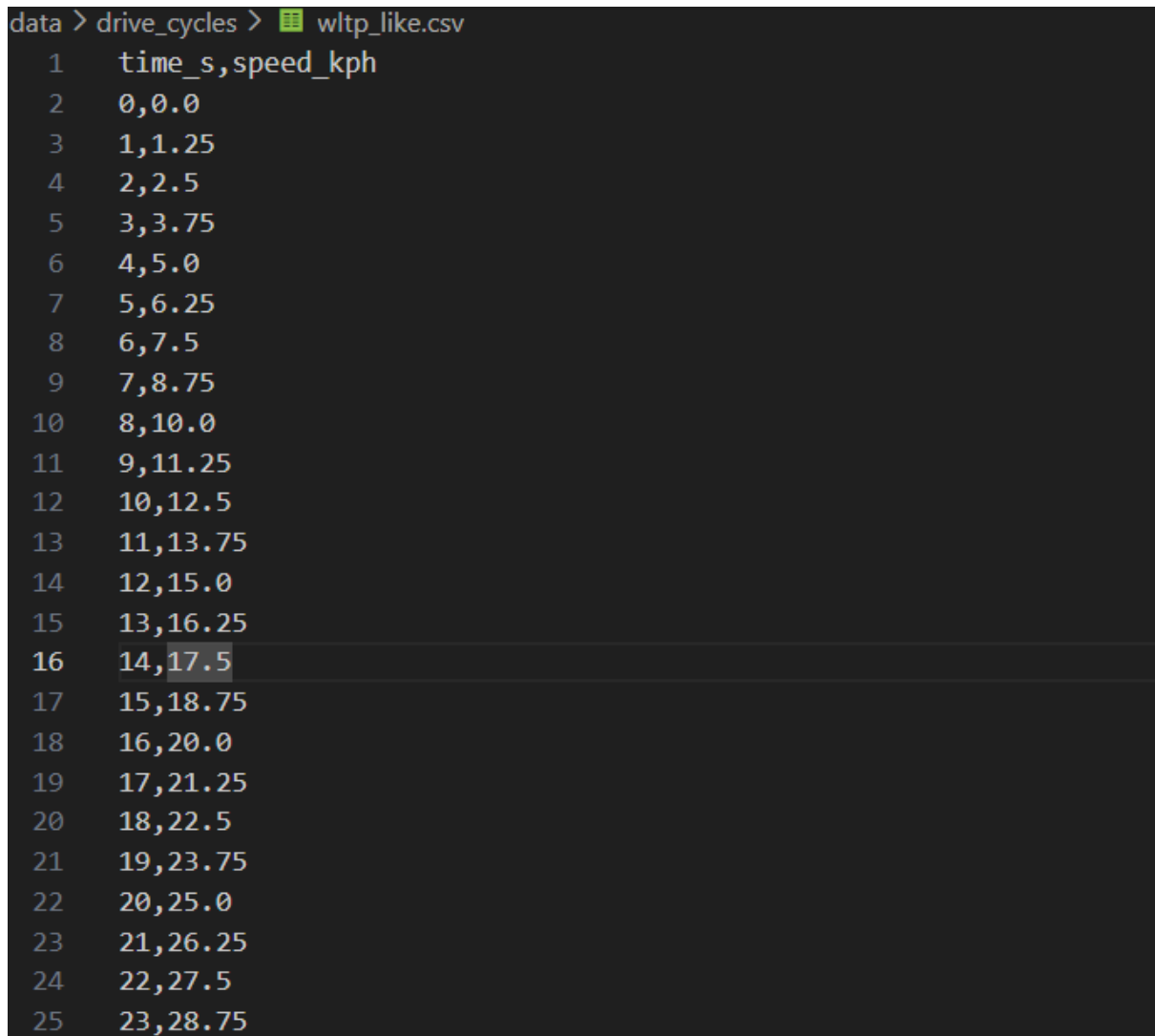
```
(.venv) PS C:\Users\Illia\Desktop\DP\project\hev_project> python -m src.simulation.simulate
>>
Saved results to: .\results\logs\run_fixed_results.csv
Saved plot to: .\results\plots\run_fixed_summary.png
Final fuel consumed (L): 0.08869655288251915

=== Energy Metrics ===
Distance driven: 6.007 km
Electrical energy consumption: 0.021 kWh/km
Total energy efficiency: 58.25 km/L_e
(.venv) PS C:\Users\Illia\Desktop\DP\project\hev_project>
```

Figure 55. Source: Screen capture of 'hev_project' './src/simulation/simulate.py' console from VS Code, created by Illia Novikov.

The resulting fuel consumption for the synthetic cycle is extremely low, only about 0.0887 liters. This tiny value is expected: the drive cycle contains only brief periods of positive power demand and very long idle sections where the engine remains off or produces minimal power. The motor performs most of the work at the beginning of the cycle, and the engine contributes primarily when SOC becomes low or when power demand temporarily increases. Overall, the results demonstrate that the rule-based controller operates as intended, using the electric motor whenever possible, protecting the battery from over-discharge, and minimizing unnecessary engine activity. The low fuel consumption therefore reflects the mild, stop-and-go nature of the synthetic cycle and the strong use of electric propulsion throughout most of the simulation. Over the 6.0 km driven, electrical energy consumption remained low at 0.021 kWh/km, yielding a high total energy efficiency of 58.25 km/L_e, which further confirms the effectiveness of the hybrid system under this mild driving pattern.

4.4.2. WLTP Cycle (Real Life-like Cycle)



```
data > drive_cycles > wltplike.csv
```

	time_s	speed_kph
1	0	0.0
2	1	1.25
3	2	2.5
4	3	3.75
5	4	5.0
6	5	6.25
7	6	7.5
8	7	8.75
9	8	10.0
10	9	11.25
11	10	12.5
12	11	13.75
13	12	15.0
14	13	16.25
15	14	17.5
16	15	18.75
17	16	20.0
18	17	21.25
19	18	22.5
20	19	23.75
21	20	25.0
22	21	26.25
23	22	27.5
24	23	28.75
25		

Figure 56. Source: Screen capture of 'hev_project' './data/drive_cycles/wltplike.csv' console from VS Code, created by Illia Novikov.

The WLTP drive cycle describes how vehicle speed changes every second to imitate real driving. It starts with a smooth acceleration from 0 to about 50 km/h, representing light urban driving. Then it holds a steady speed, followed by stronger acceleration up to 80-100 km/h to simulate suburban or short-highway conditions. After reaching the peak speed, the vehicle gradually slows down step by step until it returns to a full stop, capturing braking phases. The pattern then repeats a smaller version of the same behaviour: gentle acceleration, stable cruising at 50 km/h, and a smooth deceleration to zero. At the end, there's a short acceleration to 50 km/h and another decline back to a stop. Overall, the cycle mixes city, suburban, cruising, and braking segments, giving your simulation a realistic, variable speed profile.

WLTP Cycle Run & Results:

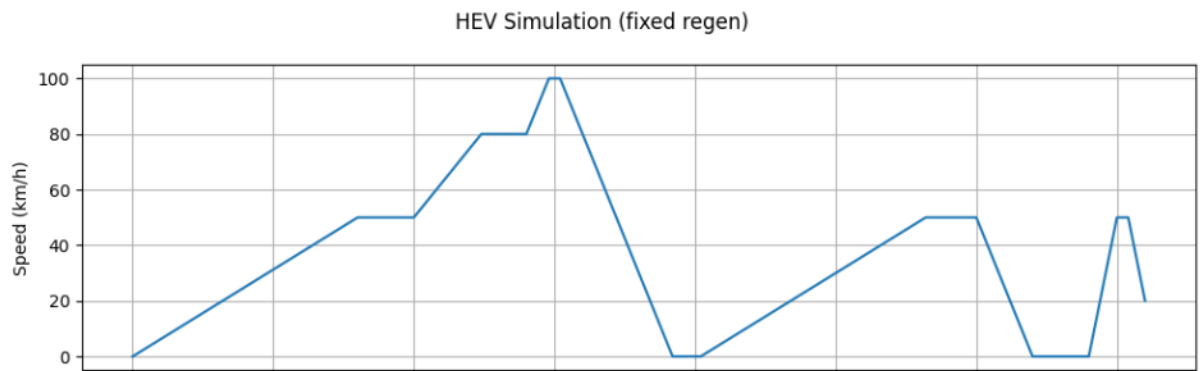


Figure 57. Source: Output plot generated by the script `'hev_project/src/simulation/simulate.py'` and exported automatically to `'./results/plots/run_fixed_summary.png'`. Created by Illia Novikov.

In this WLTP-like cycle, the vehicle experiences a much more dynamic and realistic pattern of driving compared to the synthetic test. The speed gradually rises from standstill into moderate city-like speeds around 50 km/h, then continues climbing toward highway speeds near 100 km/h.

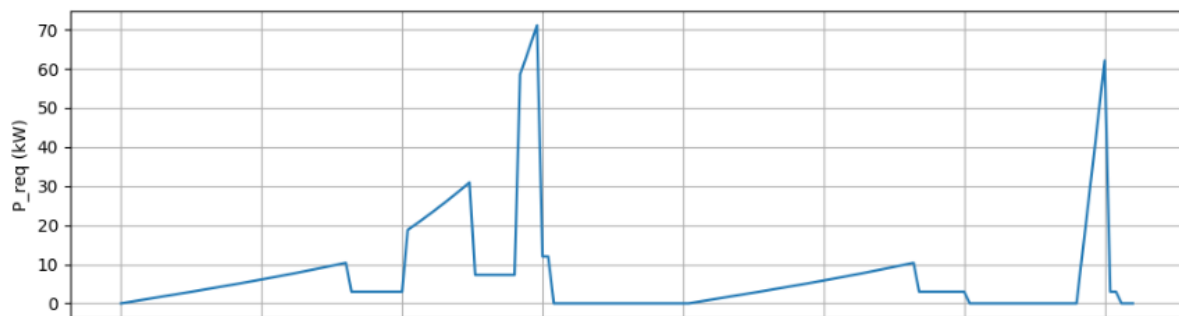


Figure 58. Source: Output plot generated by the script `'hev_project/src/simulation/simulate.py'` and exported automatically to `'./results/plots/run_fixed_summary.png'`. Created by Illia Novikov.

This long acceleration phase demands increasingly high power from the powertrain, which is visible in the large spike in required power around the peak speed. As soon as the vehicle decelerates sharply from 100 km/h down to nearly zero, the engine power drops and the system enters a low-load phase. After that, the cycle includes another gradual acceleration and cruising segment, followed by multiple smaller accelerations and decelerations that mimic suburban traffic.

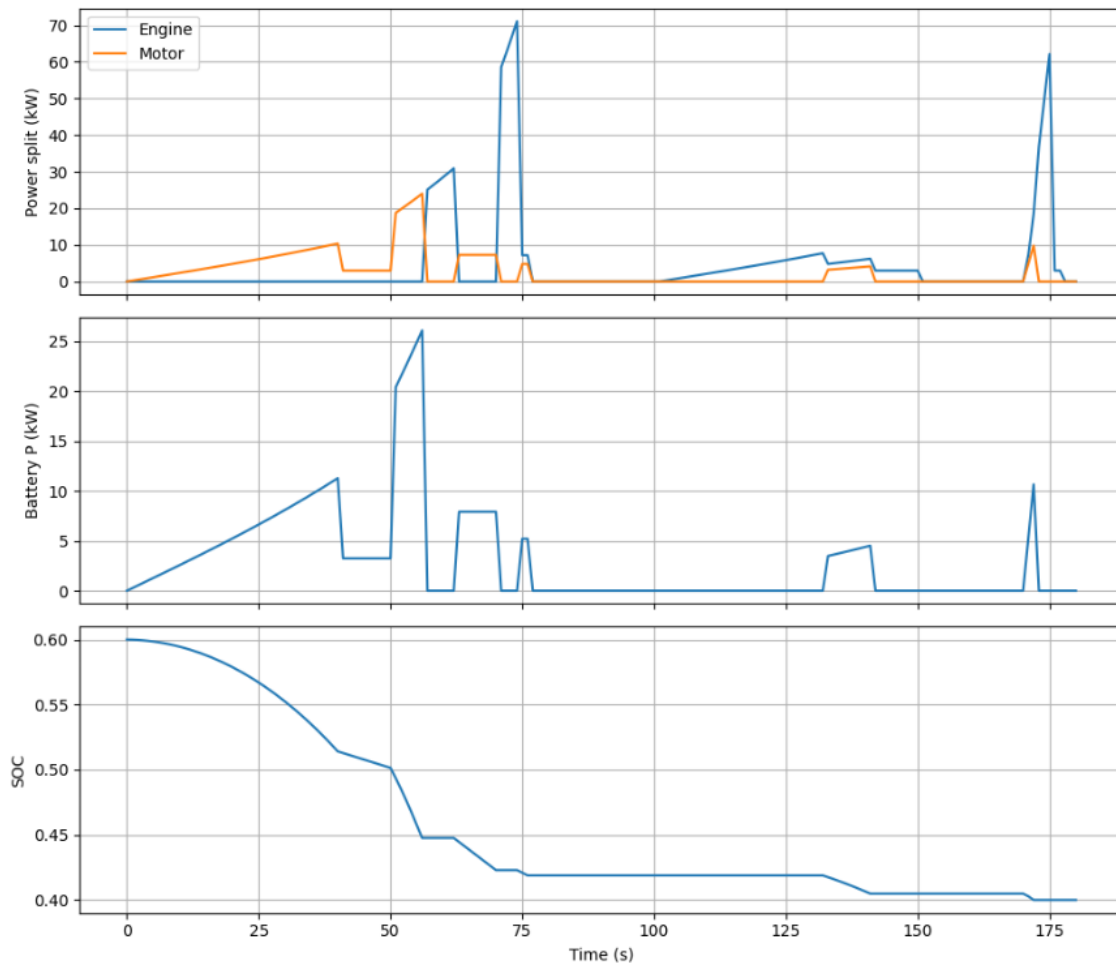


Figure 59. Source: Output plot generated by the script '`hev_project/src/simulation/simulate.py`' and exported automatically to '`./results/plots/run_fixed_summary.png`'. Created by Illia Novikov.

Because of these repeated high-power bursts and fewer long regenerative braking events, the battery steadily loses charge throughout the drive. The SOC falls from about 0.60 to roughly 0.40, showing that the vehicle relies heavily on the battery during acceleration while recovering only limited energy during braking. The engine picks up most of the power demand during high-speed segments, while the motor contributes mainly during earlier accelerations and low-speed transitions.

```
(.venv) PS C:\Users\Illia\Desktop\DP\project\hev_project> python -m src.simulation.simulate
>>
Saved results to: .\results\logs\run_fixed_results.csv
Saved plot to: .\results\plots\run_fixed_summary.png
❖ Final fuel consumed (L): 0.08720000312401593

=== Energy Metrics ===
Distance driven: 1.811 km
Electrical energy consumption: 0.079 kWh/km
Total energy efficiency: 17.52 km/L_e
(.venv) PS C:\Users\Illia\Desktop\DP\project\hev_project>
```

Figure 60. Source: Screen capture of '`hev_project`' '`./src/simulation/simulate.py`' console from VS Code, created by Illia Novikov.

Overall, the WLTP cycle's mixed urban, suburban and highway behaviour produces a more realistic pattern of power demand, battery usage, and fuel consumption, with the simulation reporting approximately 0.087 L of fuel consumed over the entire drive. In addition to fuel usage, energy-based performance metrics provide further insights. The total distance covered during the cycle was 1.81 km, over which the vehicle showed an electrical energy consumption of 0.079 kWh/km, indicating a significant contribution from the electric drivetrain. When both fuel and electrical energy were combined into a fuel-equivalent basis, the resulting total energy efficiency was 17.52 km/L_e, highlighting the effectiveness of the hybrid powertrain in converting stored energy into vehicle motion.

4.5. Comparison of Simulation Results

When comparing the synthetic 600-second cycle with the WLTP-like CSV cycle (~180 seconds), the overall fuel consumption values appear very close – 0.0887 L for the synthetic cycle and 0.0872 L for the WLTP-like run. This small difference is completely expected and reflects how both cycles activate the hybrid drivetrain in similar ways despite having different durations. The synthetic cycle is much longer, but a large portion of it consists of idling or low-speed, low-load operation where the engine remains off and the electric motor handles most of the demand. As a result, the engine runs only during a few medium-to-high-power segments, which limits overall fuel usage. The WLTP-like cycle is shorter but contains more dynamic acceleration phases and sharper speed changes, leading to brief spikes where the engine must supply higher power.

However, because the total driving time is shorter, total fuel stays close to the synthetic case. In both cycles, the SOC decreases in a similar pattern, showing that the battery supports acceleration frequently and receives only limited regeneration during deceleration. Overall, the hybrid system ends up using the engine for nearly the same amount of meaningful power output in both cycles, which explains why the final fuel consumption values are nearly identical despite the difference in cycle length.

4.6. Comparison against Manufacturer Specifications

Now let's compare the fuel-consumption results obtained from the simulation against the manufacturer specifications of the 2020 Toyota Prius Hybrid (XW50, 4th-generation facelift) to assess the realism of the model. For this vehicle, Toyota reports a combined WLTP fuel consumption of approximately 4.1-4.7 L/100 km for the 1.8 L hybrid powertrain. When this official value is scaled to short driving distances comparable to those represented by the simulated cycles, it shows a total fuel usage on the order of 0.08-0.09 L. This range closely matches the simulated fuel consumption of 0.0887 L for the 600-second synthetic cycle and 0.0872 L for the WLTP-like cycle.³²

The agreement between the simulated results and the manufacturer data can be explained by similar hybrid operating principles in both cases. In the real Prius XW50, as in the simulation, the internal combustion engine operates primarily during medium-to-high load conditions, while low-speed and low-load operation is largely supported by the electric motor, with limited regenerative braking energy recovery. As a result, overall fuel consumption is dominated by a small number of engine-on events rather than total cycle duration. Consequently, despite differences in cycle length and speed dynamics, the simulated fuel usage aligns well with Toyota's WLTP-based specifications, indicating that the developed model provides a reasonable representation of the real vehicle's fuel-efficiency behaviour under comparable driving conditions.

4.7. Sensitivity Analysis

Due to the complexity of interactions between various system components and control strategies, as well as the scope and objectives of this simulation study, a comprehensive sensitivity analysis was not performed. Key parameters such as controller thresholds – including SOC limits and power demand activation points – were fixed throughout the simulations to maintain model consistency. Additionally, changes in components, including those of the engine, motor, and battery, were not considered, although they can have a significant impact on overall system performance. The influence of different driving cycle aggressiveness levels, which could affect power demand patterns and hybrid system responses, was also excluded from this analysis. Moreover, the initial battery SOC was held constant, and it can have a potential effect on fuel consumption and energy efficiency also. For this type of rule-based hybrid vehicle simulation, focusing on baseline performance was prioritized, while acknowledging that further studies should explore these sensitivity factors to fully characterize system robustness and optimize control strategies under diverse real-world conditions.

5. Conclusion

The simulation of the smart powertrain control system for a HEV demonstrates how intelligent energy management can significantly reduce fuel consumption while maintaining stable battery operation. By evaluating both a synthetic drive cycle and a WLTP like-real-world cycle, the system consistently showed effective power distribution between the internal combustion engine and the electric motor. The control strategy successfully prioritized electric drive during low-load conditions, activated the engine only when necessary, and used regenerative braking to recover energy. As a result, the total fuel usage remained low across different driving patterns, and the SOC stayed within safe operating limits.

To conclude, the results confirm that even a simplified HEV control model can capture the essential behaviour of hybrid energy flow and demonstrate the benefits of smart power-splitting logic. The system adapts automatically to varying speed demands, reduces unnecessary engine operation, and maximizes electric contribution whenever possible. These findings highlight the potential for further refinement – such as dynamic engine on/off thresholds, predictive energy management using future speed profiles, or optimization-based control – to achieve even greater efficiency in real-world HEV applications.

³² Specs of Toyota Prius IV (XW50, facelift): <https://www.auto-data.net/en/toyota-prius-iv-xw50-facelift-2018-1.8-122hp-hybrid-e-cvt-41889>

Bibliography

1. Panday, A.; Bansal, H. A Review of Optimal Energy Management Strategies for Hybrid Electric Vehicles. *Int. J. Veh. Technol.* 2014, 2014, 160510. https://scholar.google.com/scholar_lookup?title=A+Review+of+Optimal+Energy+Management+Strategies+for+Hybrid+Electric+Vehicles&author=Panday,+A.&author=Bansal,+H.&publication_year=2014&journal=Int.+J.+Veh.+Technol.&volume=2014&pages=160510&doi=10.1155/2014/160510
2. Morfeldt, J.; Kurland, S.D.; Johansson, D.J. Carbon Footprint Impacts of Banning Cars with Internal Combustion Engines. *Transp. Res. Part D Transp. Environ.* 2021, 95, 102807. <https://www.sciencedirect.com/science/article/pii/S1361920921001115>
3. M.R.H. Mojumder.; Taher Hasan Nakib , M. Hasanuzzaman , A.S. Abdel-Khalik Advanc., challenges, and prospects of cost-effective EV: A review. 2025,128, 437-456. <https://www.sciencedirect.com/science/article/pii/S1110016825007173>
4. Ziegler, M.S.; Song, J.; Trancik, J.E. Determinants of Lithium-Ion Battery Technology Cost Decline. *Energy Environ. Sci.* 2021, 14, 6074–6098. <https://pubs.rsc.org/en/content/articlehtml/2021/xx/d1ee01313k>
5. 2022 Volkswagen Passat Technical Specifications <https://media.vw.com/assets/documents/original/14662-2022PassatTechnicalSpecsFinalv2.pdf>
6. Tesla. Tesla Model S. Available online: <https://www.tesla.com/models>
7. Specs of Volkswagen Passat <https://www.auto-data.net/en/volkswagen-passat-b8-gte-1.4-tsi-218hp-plug-in-hybrid-dsg-28003>
8. X-Engineer.org. What is the Difference between micro, Mild, Full, And Plug-in Hybrid Electric Vehicles. Available online: <https://x-engineer.org/automotive-engineering/vehicle/hybrid/micro-mild-full-hybrid-electric-vehicle/>
9. Abdellatif M. Sadeq Hybrid and Electric Vehicles - A Comprehensive Guide 2024, 10, 1398. https://www.researchgate.net/publication/379907499_Hybrid_and_Electric_Vehicles_-_A_Comprehensive_Guide

10. Ripaccioli, G.; Bernardini, D.; Di Cairano, S.; Bemporad, A.; Kolmanovsky, I.V. A Stochastic Model Predictive Control Approach for Series Hybrid Electric Vehicle Power Management. In Proceedings of the 2010 American Control Conference, Baltimore, MD, USA, 30 June 2010; pp. 5844–5849. <https://ieeexplore.ieee.org/abstract/document/5530504>
11. Zhang, F.; Wang, L.; Coskun, S.; Pang, H.; Cui, Y.; Xi, J. Energy Management Strategies for Hybrid Electric Vehicles: Review, Classification, Comparison, and Outlook. *Energies* 2020, 13, 3352. <https://www.mdpi.com/1996-1073/13/13/3352>
12. Yang, C.; Zha, M.; Wang, W.; Liu, K.; Xiang, C. Efficient Energy Management Strategy for Hybrid Electric Vehicles/Plug-In Hybrid Electric Vehicles: Review and Recent Advances under Intelligent Transportation System. *IET Intell. Transp. Syst.* 2020, 14, 702–711. <https://ietresearch.onlinelibrary.wiley.com/doi/full/10.1049/iet-its.2019.0606>
13. Jung, H. Fuel Economy of Plug-In Hybrid Electric and Hybrid Electric Vehicles: Effects of Vehicle Weight, Hybridization Ratio and Ambient Temperature. *World Electr. Veh. J.* 2020, 11, 31. <https://www.mdpi.com/2032-6653/11/2/31>
14. Jalil, N.; Kheir, N.A.; Salman, M. A Rule-Based Energy Management Strategy for a Series Hybrid Vehicle. In Proceedings of the 1997 American Control Conference, Albuquerque, NM, USA, 6 June 1997; pp. 689–693. <https://ieeexplore.ieee.org/abstract/document/611889>
15. Torreglosa, J.P.; Garcia-Triviño, P.; Vera, D.; López-García, D.A. Analyzing the Improvements of Energy Management Systems for Hybrid Electric Vehicles Using a Systematic Literature Review: How Far Are These Controls from Rule-Based Controls Used in Commercial Vehicles? *Appl. Sci.* 2020, 10, 8744. <https://www.mdpi.com/2076-3417/10/23/8744>
16. Jane, R.S.; James, C.; Kim, T. Using AI-ML to Develop Energy and Exergy Flow Characterizations for Multi-Domain Operations. In Proceedings of the Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications, Baltimore, MD, USA, 12 April 2021; Volume 11746, p. 1174628. <https://www.spiedigitallibrary.org/conference-proceedings-of-spie/11746/1174628/Using-AI-ML-to-develop-energy-and-exergy-flow-characterizations/10.1117/12.2585775.short>
17. Nenadic, N.G.; Trabold, T.A.; Thurston, M.G. Cell Replacement Strategies for Lithium-Ion Battery Packs. *Batteries* 2020, 6, 39. <https://www.mdpi.com/2313-0105/6/3/39>
18. Li, D.; Zhao, D.; Zhang, Q.; Chen, Y. Reinforcement Learning and Deep Learning Based Lateral Control for Autonomous Driving [Application Notes]. *IEEE Comput. Intell. Mag.* 2019, 14, 83–98. <https://ieeexplore.ieee.org/abstract/document/8686348>

19. Brake Specific Fuel Consumption (BSFC): <https://x-engineer.org/brake-specific-fuel-consumption-bsfc/>
20. Electric Vehicle Motor: <https://x-engineer.org/electric-vehicle-motor-torque-power-curves/>
21. What is a battery electric vehicle: <https://x-engineer.org/battery-electric-vehicle-bev/>
22. Anna Tomaszewska, Zhengyu Chu, Xuning Feng , Simon O'Kane, Xinhua Liu, Jingyi Chen, Chenzhen Ji, Elizabeth Endler, Ruihe Li, Lishuo Liu, Yalun Li , Siqi Zheng , Sebastian Vetterlein , Ming Gao , Jiuyu Du , Michael Parkes, Minggao Ouyang, Monica Marinescu, Gregory Offer, Billy Wu Lithium-ion battery fast charging: A review 2019, 1, 100011. <https://www.sciencedirect.com/science/article/pii/S2590116819300116>
23. Tire model for longitudinal forces: <https://x-engineer.org/tire-model-longitudinal-forces/>
24. Ed. Louback, A. Biswas, Fab. Machado, Ali Emadi A review of the design process of energy management systems for dual-motor battery electric vehicles 2024, 193, 114293. <https://www.sciencedirect.com/science/article/pii/S1364032124000169>
25. Ah. K. Abbas, Raz. Ayop Advanced energy-management and sizing techniques for renewable microgrids with electric-vehicle integration: A review 2025, 27, 106252. <https://www.sciencedirect.com/science/article/pii/S2590123025023242>
26. Theo Hofman, Alex Serrarens A Rule-based energy management strategies for hybrid vehicles 2007, 150, 014448. https://www.researchgate.net/publication/228679078_A_Rule-based_energy_management_strategies_for_hybrid_vehicles
27. Farzad Raj. Salmasi Control Strategies for Hybrid Electric Vehicles: Evolution, Classification, Comparison, and Future Trends 2007, 109, 899933. https://www.researchgate.net/publication/271526695_Control_Strategies_for_Hybrid_Electric_Vehicles_Evolution_Classification_Comparison_and_Future_Trends
28. Vaishnavi Tella, Mohamad Alzayed A Comprehensive Review of Energy Management Strategies in Hybrid Electric Vehicles: Comparative Analysis and Challenges 2024, 109, 509737. https://www.researchgate.net/publication/386360232_A_Comprehensive_Review_of_Energy_Management_Strategies_in_Hybrid_Electric_Vehicles_Comparative_Analysis_and_Challenges

29. H. Tarout,, H. Zaki, A. Chahbouni, E. Ennajih, El M. Louragli Optimizing Energy Consumption in Electric Vehicles: A Systematic and Bibliometric Review of Recent Advances 2025, 577, 100577. <https://www.mdpi.com/2032-6653/16/10/577>
30. Vispi Karkaria, Ying-Kuan Tsai An optimization-centric review on integrating artificial intelligence and digital twin technologies in manufacturing 2025, 161, 243420. <https://www.tandfonline.com/doi/full/10.1080/0305215X.2024.2434201>
31. Elia Grano, Manfredi Villani, Manfredi Villani Are We Testing Vehicles the Right Way? Challenges of Electrified and Connected Vehicles for Standard Drive Cycles and On-Road Testing 2025, 94, 020094. <https://www.mdpi.com/2032-6653/16/2/94>
32. Specs of Toyota Prius IV (XW50, facelift): <https://www.auto-data.net/en/toyota-prius-iv-xw50-facelift-2018-1.8-122hp-hybrid-e-cvt-41889>